Table of contents

1 Linear Algebra **1**

1.1 Intro to Numerical Linear Algebra 1
1.2 Notation 2
1.3 Vectors and Matrices in Python 3
1.4 Matrix and Vector Operations 7
1.4.1 The Dot Product 7
1.4.2 Matrix Multiplication 9
1.5 The LU Factorization 14
1.5.1 A Recap of Row Reduction 14
1.5.2 The LU Decomposition 17
1.5.3 Solving Triangular Systems 23
1.5.4 Solving Systems with LU Decomposition 28
1.6 The QR Factorization 29
1.7 Over-determined Systems and Curve Fitting 39
1.8 The Eigenvalue-Eigenvector Problem 42
1.9 Algorithm Summaries 48
1.10 Problems 49
1.11 Projects 59
1.11.1 The Google Page Rank Algorithm 59
1.11.2 Alternative Methods For Solving $Ax = b$ 66

1 Linear Algebra

You cannot learn too much linear algebra.
– Every mathematician

1.1 Intro to Numerical Linear Algebra

The preceding comment says it all – linear algebra is the most important of all of the mathematical tools that you can learn as a practitioner of the mathematical sciences. The theorems, proofs, conjectures, and big ideas in almost every other mathematical field find their roots in linear algebra. Numerical Linear Algebra is the study of algorithms for solving problems in linear algebra. This subject has a somewhat different flavour than the numerical analysis we are studying in the main text and hence we present it as an optional appendix.

Our goal in this appendix is to explore numerical algorithms for the primary questions of linear algebra:

- solving systems of equations,
- finding eigenvalue-eigenvector pairs for a matrix.

Take careful note that in our current digital age, numerical linear algebra and its fast algorithms are behind the scenes for wide varieties of computing applications. Applications of numerical linear algebra include:

- building neural networks and AI algorithms,
- determining the most important web page in a Google search,
- modelling realistic 3D environments in video games,
- digital image processing,
- and many many more.

What's more, researchers have found provably optimal ways to perform most of the typical tasks of linear algebra so most scientific software works very well and very quickly with linear algebra.

1.2 Notation

Throughout this chapter we will use the following notation conventions.

- A bold mathematical symbol such as x or u will represent a vector.
- If u is a vector then u_j will be the jth entry of the vector.
- Vectors will typically be written vertically with parenthesis as delimiters such as

$$u = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}. \tag{1.1}$$

- Two bold symbols separated by a centred dot such as $u \cdot v$ will represent the dot product of two vectors.
- A capital mathematical symbol such as A or X will represent a matrix
- If A is a matrix then A_{ij} will be the element in the ith row and j^{th} column of the matrix.
- A matrix will typically be written with parenthesis as delimiters such as

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & \pi \end{pmatrix}. \tag{1.2}$$

- The juxtaposition of a capital symbol and a bold symbol such as Ax will represent matrix-vector multiplication.
- A lower case or Greek mathematical symbol such as x, c, or λ will represent a scalar.
- The scalar field of real numbers is given as $\mathbb{R}$ and the scalar field of complex numbers is given as $\mathbb{C}$.
- The symbol $\mathbb{R}^n$ represents the collection of all n-dimensional vectors where the elements are drawn from the real numbers.
- The symbol $\mathbb{C}^n$ represents the collection of all n-dimensional vectors where the elements are drawn from the complex numbers.

It is an important part of learning to read and write linear algebra to give special attention to the symbolic language so you can communicate your work easily and efficiently.

1.3 Vectors and Matrices in Python

We first need to understand how Python's `numpy` library builds and stores vectors and matrices. The following exercises will give you some experience building and working with these data structures and will point out some common pitfalls that mathematicians fall into when using Python for linear algebra.

Example 1.3.1 (numpy Arrays)**.** In Python you can build a list using square brackets such as `[1,2,3]`. This is called a "Python list" and is NOT a vector in the way that we think about it mathematically. It is simply an ordered collection of objects. To build mathematical vectors in Python we need to use `numpy` arrays with `np.array()`. For example, the vector

$$u = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} \tag{1.3}$$

would be built with the following code.

```
import numpy as np
u = np.array([1,2,3])
print(u)
```

Notice that Python defines the vector `u` as a matrix with only one dimension. You can see that in the following code.

```
import numpy as np
u = np.array([1,2,3])
print("The length of the u vector is \n",len(u))
print("The shape of the u vector is \n",u.shape)
```

Example 1.3.2 (numpy Matrices)**.** In `numpy`, a matrix is validly built as a list of lists. For example, the matrix

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} \tag{1.4}$$

is defined using `np.array()` where each row is an individual list, and the matrix is a collection of these lists.

```
import numpy as np
A = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(A)
```

Moreover, we can extract the shape, the number of rows, and the number of columns of A using the `A.shape` command. To be a bit more clear on this one we will use the matrix

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \tag{1.5}$$

```
import numpy as np
A = np.array([[1,2,3],[4,5,6]])
print("The shape of the A matrix is \n",A.shape)
print("Number of rows in A is \n",A.shape[0])
print("Number of columns in A is \n",A.shape[1])
```

Example 1.3.3 (Row and Column Vectors in Python)**.** You can more specifically build row or column vectors in Python using the `np.array()` command and then only specifying one row or column. But take careful note that `numpy` treats a 1D array (like `np.array([1,2,3])`) differently from a 2D array (like `np.array([[1,2,3]])`). For example, if you want the vectors

$$u = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} \quad \text{and} \quad v = \begin{pmatrix} 4 & 5 & 6 \end{pmatrix} \tag{1.6}$$

then we would use the following Python code.

```
import numpy as np
u = np.array([[1],[2],[3]])
print("The column vector u is \n",u)
v = np.array([[1,2,3]])
print("The row vector v is \n",v)
```

Alternatively, if you want to define a column vector you can define a row vector (since there are far fewer brackets to keep track of) and then transpose the matrix to turn it into a column. Note that the `.transpose()` method (or `.T`) only swaps dimensions; it won't turn a 1D array into a 2D column vector unless it was already 2D.

```
import numpy as np
u = np.array([[1,2,3]])
u = u.transpose()
print("The column vector u is \n",u)
```

Example 1.3.4 (Matrix Indexing)**.** Python indexes all arrays, vectors, lists, and matrices starting from index 0. Let us get used to this fact.

Consider the matrix A defined in the previous problem. Mathematically we know that the entry in row 1 column 1 is a 1, the entry in row 1 column 2 is a 2, and so on. However, with Python we need to shift the way that we enumerate the rows and columns of a matrix. Hence we would say that the entry in row 0 column 0 is a 1, the entry in row 0 column 1 is a 2, and so on.

Mathematically we can view all Python matrices as follows. If A is an $n \times n$ matrix then

$$A = \begin{pmatrix} A_{0,0} & A_{0,1} & A_{0,2} & \cdots & A_{0,n-1} \\ A_{1,0} & A_{1,1} & A_{1,2} & \cdots & A_{1,n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ A_{n-1,0} & A_{n-1,1} & A_{n-1,2} & \cdots & A_{n-1,n-1} \end{pmatrix} \tag{1.7}$$

Similarly, we can view all vectors as follows. If u is an $n \times 1$ vector then

$$u = \begin{pmatrix} u_0 \\ u_1 \\ \vdots \\ u_{n-1} \end{pmatrix} \tag{1.8}$$

The following code should help to illustrate this indexing convention.

```
import numpy as np
A = np.array([[1,2,3],[4,5,6],[7,8,9]])
print("Entry in row 0 column 0 is",A[0,0])
print("Entry in row 0 column 1 is",A[0,1])
print("Entry in the bottom right corner",A[2,2])
```

Exercise 1.3.1. Build your own matrix in Python and practice choosing individual entries from the matrix.

Example 1.3.5 (Matrix Slicing)**.** The last thing that we need to be familiar with is *slicing* a matrix. The term "slicing" generally refers to pulling out individual rows, columns, entries, or blocks from a list, array, or matrix in Python. Examine the code below to see how to slice parts out of a `numpy` matrix.

```
import numpy as np
A = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(A)
print("The first column of A is \n",A[:,0])
print("The second row of A is \n",A[1,:])
print("The top left 2x2 sub matrix of A is \n",A[:-1,:-1])
print("The bottom right 2x2 sub matrix of A is \n",A[1:,1:])
u = np.array([1,2,3,4,5,6])
print("The first 3 entries of the vector u are \n",u[:3])
print("The last entry of the vector u is \n",u[-1])
print("The last two entries of the vector u are \n",u[-2:])
```

Exercise 1.3.2. Define the matrix A and the vector u in Python. Then perform all of the tasks below.

$$A = \begin{pmatrix} 1 & 3 & 5 & 7 \\ 2 & 4 & 6 & 8 \\ -3 & -2 & -1 & 0 \end{pmatrix} \quad \text{and} \quad u = \begin{pmatrix} 10 \\ 20 \\ 30 \end{pmatrix} \tag{1.9}$$

1. Print the matrix A, the vector u, the shape of A, and the shape of u.
2. Print the first column of A.
3. Print the first two rows of A.
4. Print the first two entries of u.
5. Print the last two entries of u.
6. Print the bottom left 2×2 submatrix of A.
7. Print the middle two elements of the middle row of A.

1.4 Matrix and Vector Operations

Now let us start doing some numerical linear algebra. We start our discussion with the basics: the dot product and matrix multiplication. The numerical routines in Python's `numpy` packages are designed to do these tasks in very efficient ways but it is a good coding exercise to build your own dot product and matrix multiplication routines just to further cement the way that Python deals with these data structures and to remind you of the mathematical algorithms. What you will find in numerical linear algebra is that the indexing and the housekeeping in the codes is the hardest part. So why do not we start "easy."

1.4.1 The Dot Product

Exercise 1.4.1. This problem is meant to jog your memory about dot products, how to compute them, and what you might use them for. If your linear algebra is a bit rusty then read ahead a bit and then come back to this problem.

Consider two vectors u and v defined as

$$u = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \quad \text{and} \quad v = \begin{pmatrix} 3 \\ 4 \end{pmatrix}. \tag{1.10}$$

1. Draw a picture showing both u and v.
2. What is $u \cdot v$?
3. What is $\|u\|$?
4. What is $\|v\|$?
5. What is the angle between u and v?
6. Give two reasons why we know that u is not perpendicular to v.
7. What is the scalar projection of u onto v? Draw this scalar projections on your picture from part (1).
8. What is the scalar projection of v onto u? Draw this scalar projections on your picture from part (1).

Now let us get the formal definitions of the dot product on the table.

Definition 1.4.1 (Dot product)**.** The **dot product** of two vectors $u, v \in \mathbb{R}^n$ is

$$u \cdot v = \sum_{j=1}^{n} u_j v_j. \tag{1.11}$$

Without summation notation the dot product of two vectors is ,

$$u \cdot v = u_1 v_1 + u_2 v_2 + \cdots + u_n v_n. \tag{1.12}$$

You may also recall that the dot product of two vectors is given geometrically as

$$u \cdot v = \|u\| \|v\| \cos\theta \tag{1.13}$$

where $\|u\|$ and $\|v\|$ are the magnitudes (or lengths) of u and v respectively, and θ is the angle between the two vectors. In physical applications the dot product is often used to find the angle between two vectors (e.g. between two forces). Hence, the last form of the dot product is often rewritten as

$$\theta = \cos^{-1}\left(\frac{u \cdot v}{\|u\| \|v\|}\right). \tag{1.14}$$

Definition 1.4.2 (Magnitude of a Vector)**.** The **magnitude** of a vector $u \in \mathbb{R}^n$ is defined as [1]

$$\|u\| = \sqrt{u \cdot u}. \tag{1.15}$$

Exercise 1.4.2. Write a Python function that accepts two vectors (defined as `numpy` arrays) and returns the dot product. Write this code without the use any loops.

```
import numpy as np
def myDotProduct(u,v):
    return # the dot product formula uses a product inside a sum.
```

---

[1] You should also note that $\|u\| = \sqrt{u \cdot u}$ is not the only definition of distance. More generally, if you let $\langle u, v \rangle$ be an inner product for u and v in some vector space $\mathcal{V}$ then $\|u\| = \sqrt{\langle u, u \rangle}$. In most cases in this text we will be using the dot product as our preferred inner product so we will not have to worry much about this particular natural extension of the definition of the length of a vector.

Exercise 1.4.3. Test your `myDotProduct()` function on several dot products to make sure that it works. Example code to find the dot product between

$$u = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} \quad \text{and} \quad v = \begin{pmatrix} 4 \\ 5 \\ 6 \end{pmatrix} \tag{1.16}$$

is given below. Test your code on other vectors. Then implement an error catch into your code to catch the case where the two input vectors are not the same size. You will want to use the `len()` command to find the length of the vectors.

```
u = np.array([1,2,3])
v = np.array([4,5,6])
myDotProduct(u,v)
```

Exercise 1.4.4. Try sending Python lists instead of `numpy` arrays into your `myDotProduct` function. What happens? Why does it happen? What is the cautionary tale here? Modify your `myDotProduct()` function one more time so that it starts by converting the input vectors into `numpy` arrays.

```
u = [1,2,3]
v = [4,5,6]
myDotProduct(u,v)
```

Exercise 1.4.5. The `numpy` library in Python has a built-in command for doing the dot product: `np.dot()`. Test the `np.dot()` command and be sure that it does the same thing as your `myDotProduct()` function.

1.4.2 Matrix Multiplication

Next we will blow the dust off of your matrix multiplication skills.

Exercise 1.4.6. Verify that the product of A and B is indeed what we show below. Work out all of the details by hand.

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix} \qquad B = \begin{pmatrix} 7 & 8 & 9 \\ 10 & 11 & 12 \end{pmatrix} \tag{1.17}$$

$$AB = \begin{pmatrix} 27 & 30 & 33 \\ 61 & 68 & 75 \\ 95 & 106 & 117 \end{pmatrix} \tag{1.18}$$

Now that you have practised the algorithm for matrix multiplication we can formalize the definition and then turn the algorithm into a Python function.

Definition 1.4.3 (Matrix Multiplication)**.** If A and B are matrices with $A \in \mathbb{R}^{n\times p}$ and $B \in \mathbb{R}^{p\times m}$ then the product AB is defined as

$$(AB)_{ij} = \sum_{k=1}^{p} A_{ik}B_{kj}. \tag{1.19}$$

A moment's reflection reveals that each entry in the matrix product is actually a dot product,

$$(\text{Entry in row } i \text{ column } j \text{ of } AB) = (\text{Row } i \text{ of matrix } A) \cdot (\text{Column } j \text{ of matrix } B)\,. \tag{1.20}$$

Exercise 1.4.7. The definition of matrix multiplication above contains the cryptic phrase *a moment's reflection reveals that each entry in the matrix product is actually a dot product.* Let us go back to the matrices A and B defined in Exercise 1.4.6 above and re-evaluate the matrix multiplication algorithm to make sure that you see each entry as the end result of a dot product.

We want to find the product of matrices A and B using dot products.

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix} \qquad B = \begin{pmatrix} 7 & 8 & 9 \\ 10 & 11 & 12 \end{pmatrix} \tag{1.21}$$

1. Why will the product AB clear be a 3×3 matrix?

2. When we do matrix multiplication we take the product of a row from the first matrix times a column from the second matrix … at least that's how many people think of it when they perform the operation by hand.

   a. The rows of A can be written as the vectors

$$a_0 = \begin{pmatrix} 1 & 2 \end{pmatrix} \tag{1.22}$$

$$a_1 = (\underline{\qquad\quad} \quad \underline{\qquad\quad}) \tag{1.23}$$

$$a_2 = (\underline{\qquad\quad} \quad \underline{\qquad\quad}) \tag{1.24}$$

   b. The columns of B can be written as the vectors

$$b_0 = \begin{pmatrix} 7 \\ 10 \end{pmatrix} \tag{1.25}$$

$$b_1 = \begin{pmatrix} \underline{\qquad\quad} \\ \underline{\qquad\quad} \end{pmatrix} \tag{1.26}$$

$$b_2 = \begin{pmatrix} \underline{\qquad\quad} \\ \underline{\qquad\quad} \end{pmatrix} \tag{1.27}$$

3. Now let us write each entry in the product AB as a dot product.

$$AB = \begin{pmatrix} a_0 \cdot b_0 & \underline{\quad} \cdot \underline{\quad} & \underline{\quad} \cdot \underline{\quad} \\ \underline{\quad} \cdot \underline{\quad} & \underline{\quad} \cdot \underline{\quad} & \underline{\quad} \cdot \underline{\quad} \\ \underline{\quad} \cdot \underline{\quad} & \underline{\quad} \cdot \underline{\quad} & \underline{\quad} \cdot \underline{\quad} \end{pmatrix} \tag{1.28}$$

4. Verify that you get

$$AB = \begin{pmatrix} 27 & 30 & 33 \\ 61 & 68 & 75 \\ 95 & 106 & 117 \end{pmatrix} \tag{1.29}$$

   when you perform all of the dot products from part (3).

Exercise 1.4.8. The observation that matrix multiplication is just a bunch of dot products is what makes the code for doing matrix multiplication very fast and very streamlined. We want to write a Python function that accepts two `numpy` matrices and returns the product of the two matrices. Inside the code we will leverage the `np.dot()` command to do the appropriate dot products.

Partial code is given below. Fill in all of the details and give ample comments showing what each line does.

```
import numpy as np
def myMatrixMult(A,B):
    # Get the shapes of the matrices A and B.
    # Then write an if statement that catches size mismatches
    # in the matrices.  Next build a zeros matrix that is the
    # correct size for the product of A and B.
    AB = ???
    # AB is a zeros matix that will be filled with the values
    # from the product
    #
    # Next we do a double for-loop that loops through all of
    # the indices of the product
    for i in range(n): # loop over the rows of AB
        for j in range(m): # loop over the columns of AB
            # use the np.dot() command to take the dot product
            AB[i,j] = ???
    return AB
```

Use the following test code to determine if you actually get the correct matrix product out of your code.

```
``` python
A = np.array([[1,2],[3,4],[5,6]])
B = np.array([[7,8,9],[10,11,12]])
AB = myMatrixMult(A,B)
print(AB)
```

Exercise 1.4.9. Try your `myMatrixMult()` function on several other matrix multiplication problems.

```

Exercise 1.4.10. Build in an error catch so that your `myMatrixMult()` function catches when the input matrices do not have compatible sizes for multiplication. Write your code so that it returns an appropriate error message in this special case.

Now that you have been through the exercise of building a matrix multiplication function we will admit that using it inside larger coding problems would be a bit cumbersome (and perhaps annoying). It would be nice to just type `@` and have Python just *know* that you mean to do matrix multiplication. This is where `numpy` arrays come in quite handy.

Exercise 1.4.11 (Matrix Multiplication with Python)**.** Python will handle matrix multiplication easily so long as the matrices are defined as `numpy` arrays. For example, with the matrices A and B from above if you can just type `A @ B` (or `np.dot(A, B)`) in Python and you will get the correct result. Pretty nice!! Let us take another moment to notice, though, that regular Python lists do not behave in the same way. Can you guess what happens if you run the following Python code? (Note: `*` does strictly element-wise multiplication for numpy arrays, which we will see in a moment).

```
A = [[1,2],[3,4],[5,6]] # a Python list of lists
B = [[7,8,9],[10,11,12]] # a Python list of lists
A @ B
```

Example 1.4.1 (Element-by-Element Multiplication)**.** Sometimes it is convenient to do naive multiplication of matrices when you code. That is, if you have two matrices that are the same size, "naive multiplication" would just line up the matrices on top of each other and multiply the corresponding entries.[2] In Python you can do this with the `*` operator (or `np.multiply()`). The code below demonstrates this tool with the matrices

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix} \quad \text{and} \quad B = \begin{pmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{pmatrix}. \tag{1.30}$$

[2] You might have thought that *naive multiplication* was a much more natural way to do matrix multiplication when you first saw it. Hopefully now you see the power in the definition of matrix multiplication that we actually use. If not, then I give you this moment to ponder that (a) matrix multiplication is just a bunch of dot products, and (b) dot products can be seen as projections. Hence, matrix multiplication is really just a projection of the rows of A onto the columns of B. This has much more rich geometric flavour than *naive multiplication*.

(Note that the product AB does not make sense under the mathematical definition of matrix multiplication, but it does make sense in terms of element-by-element ("naive") multiplication.)

```
import numpy as np
A = np.array([[1,2],[3,4],[5,6]])
B = np.array([[7,8],[9,10],[11,12]])
print(A * B)
```

The key takeaways for doing matrix multiplication in Python are as follows:

- If you are doing linear algebra in Python then you should define vectors and matrices with `np.array()`.
- If your matrices are defined with `np.array()` then `@` (or `np.dot()`) does regular matrix multiplication and `*` does element-by-element multiplication.

1.5 The LU Factorization

One of the many classic problems of linear algebra is to solve the linear system $Ax = b$ where A is a matrix of coefficients and b is a vector of right-hand sides. You likely recall your go-to technique for solving systems was row reduction (or Gaussian Elimination). Furthermore, you likely rarely actually did row reduction by hand, and instead you relied on a computer to do most of the computations for you. Just what was the computer doing, exactly? Do you think that it was actually following the same algorithm that you did by hand?

1.5.1 A Recap of Row Reduction

Let us blow the dust off your row reduction skills before we look at something better.

Exercise 1.5.1. Solve the following system of equations by hand.

$$\begin{aligned} x_0 + 2x_1 + 3x_2 &= 1 \\ 4x_0 + 5x_1 + 6x_2 &= 0 \\ 7x_0 + 8x_1 &= 2 \end{aligned} \tag{1.31}$$

Note that the system of equations can also be written in the matrix form

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 2 \end{pmatrix} \tag{1.32}$$

If you need a nudge to get started then jump ahead to the next problem.

Exercise 1.5.2. We want to solve the system of equations

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 2 \end{pmatrix} \tag{1.33}$$

Row Reduction Process:

Note: Throughout this discussion we use Python-type indexing so the rows and columns are enumerated starting at 0. That is to say, we will talk about row 0, row 1, and row 2 of a matrix instead of rows 1, 2, and 3.

1. Augment the coefficient matrix and the vector on the right-hand side to get

$$\left(\begin{array}{ccc|c} 1 & 2 & 3 & 1 \\ 4 & 5 & 6 & 0 \\ 7 & 8 & 0 & 2 \end{array}\right) \tag{1.34}$$

2. The goal of row reduction is to perform elementary row operations until our augmented matrix gets to (or at least gets as close as possible to)

$$\left(\begin{array}{ccc|c} 1 & 0 & 0 & \star \\ 0 & 1 & 0 & \star \\ 0 & 0 & 1 & \star \end{array}\right) \tag{1.35}$$

   The allowed elementary row operations are:

   a. We are allowed to scale any row.

   b. We can add two rows.

c. We can interchange two rows.

3. We are going to start with column 0. We already have the "1" in the top left corner so we can use it to eliminate all of the other values in the first column of the matrix.

   a. For example, if we multiply the 0th row by -4 and add it to the first row we get

$$\left(\begin{array}{ccc|c} 1 & 2 & 3 & 1 \\ 0 & -3 & -6 & -4 \\ 7 & 8 & 0 & 2 \end{array}\right). \tag{1.36}$$

   b. Multiply row 0 by a scalar and add it to row 2. Your end result should be

$$\left(\begin{array}{ccc|c} 1 & 2 & 3 & 1 \\ 0 & -3 & -6 & -4 \\ 0 & -6 & -21 & -5 \end{array}\right). \tag{1.37}$$

   What did you multiply by? Why?

4. Now we should deal with column 1.

   a. We want to get a 1 in row 1 column 1. We can do this by scaling row 1. What did you scale by? Why? Your end result should be

$$\left(\begin{array}{ccc|c} 1 & 2 & 3 & 1 \\ 0 & 1 & 2 & \frac{4}{3} \\ 0 & -6 & -21 & -5 \end{array}\right). \tag{1.38}$$

   b. Now scale row 1 by something and add it to row 0 so that the entry in row 0 column 1 becomes a 0.

   c. Next scale row 1 by something and add it to row 2 so that the entry in row 2 column 1 becomes a 0.

   d. At this point you should have the augmented system

$$\left(\begin{array}{ccc|c} 1 & 0 & -1 & -\frac{5}{3} \\ 0 & 1 & 2 & \frac{4}{3} \\ 0 & 0 & -9 & 3 \end{array}\right). \tag{1.39}$$

5. Finally we need to work with column 2.

   a. Make the value in row 2 column 2 a 1 by scaling row 2. What did you scale by? Why?

   b. Scale row 2 by something and add it to row 1 so that the entry in row 1 column 2 becomes a 0. What did you scale by? Why?

c. Scale row 2 by something and add it to row 0 so that the entry in row 0 column 2 becomes a 0. What did you scale by? Why?

d. By the time you have made it this far you should have the system

$$\left(\begin{array}{ccc|c} 1 & 0 & 0 & -2 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & -\frac{1}{3} \end{array}\right) \tag{1.40}$$

and you should be able to read off the solution to the system.

6. You should verify your answer in two different ways:

a. If you substitute your values into the original system then all of the equal signs should be true. Verify this.

b. If you substitute your values into the matrix equation and perform the matrix-vector multiplication on the left-hand side of the equation you should get the right-hand side of the equation. Verify this.

Exercise 1.5.3. Summarize the process for doing Gaussian Elimination to solve a square system of linear equations.

1.5.2 The LU Decomposition

You may have used the `rref()` command either on a calculator in other software to perform row reduction in the past. You will be surprised to learn that there is no `rref()` command in Python's `numpy` library! That's because there are far more efficient and stable ways to solve a linear system on a computer. There is an `rref` command in Python's `sympy` (symbolic Python) library, but given that it works with symbolic algebra it is quite slow.

In solving systems of equations we are interested in equations of the form $Ax = b$. Notice that the b vector is just along for the ride, so to speak, in the row reduction process since none of the values in b actually cause you to make different decisions in the row reduction algorithm. Hence, we only really need to focus on the matrix A. Furthermore, let us change our awfully restrictive view of always seeking a matrix of the form

$$\left(\begin{array}{cccc|c} 1 & 0 & \cdots & 0 & \star \\ 0 & 1 & \cdots & 0 & \star \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & \star \end{array}\right) \tag{1.41}$$

and instead say:

What if we just row reduce until the system is simple enough to solve by hand?

That's what the next several exercises are going to lead you to. Our goal here is to develop an algorithm that is fast to implement on a computer and simultaneously performs the same basic operations as row reduction for solving systems of linear equations.

Exercise 1.5.4. Let A be defined as

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{pmatrix}. \tag{1.42}$$

1. The first step in row reducing A would be to multiply row 0 by -4 and add it to row 1. Do this operation by hand so that you know what the result is supposed to be. Check out the following amazing observation. Define the matrix L_1 as follows:

$$L_1 = \begin{pmatrix} 1 & 0 & 0 \\ -4 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}. \tag{1.43}$$

   Now multiply L_1 and A.

$$L_1 A = \begin{pmatrix} __ & __ & __ \\ __ & __ & __ \\ __ & __ & __ \end{pmatrix} \tag{1.44}$$

   What just happened?!

2. Let us do it again. The next step in the row reduction of your result from part (1) would be to multiply row 0 by -7 and add to row 2. Again, do this by hand so you know what the result should be. Then define the matrix L_2 as

$$L_2 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -7 & 0 & 1 \end{pmatrix} \tag{1.45}$$

   and find the product $L_2 (L_1 A)$.

$$L_2 (L_1 A) = \begin{pmatrix} __ & __ & __ \\ __ & __ & __ \\ __ & __ & __ \end{pmatrix} \tag{1.46}$$

   Pure insanity!!

3. Now let us say that you want to make the entry in row 2 column 1 into a 0 by scaling row 1 by something and then adding to row 2. Determine what the scalar would be and then determine which matrix, call it L_3, would do the trick so that $L_3(L_2L_1A)$ would be the next row reduced step.

$$L_3 = \begin{pmatrix} 1 & ___ & ___ \\ ___ & 1 & ___ \\ ___ & ___ & 1 \end{pmatrix} \tag{1.47}$$

$$L_3\left(L_2L_1A\right) = \begin{pmatrix} ___ & ___ & ___ \\ ___ & ___ & ___ \\ ___ & ___ & ___ \end{pmatrix} \tag{1.48}$$

Exercise 1.5.5. Apply the same idea from the previous problem to do the first three steps of row reduction to the matrix

$$A = \begin{pmatrix} 2 & 6 & 9 \\ -6 & 8 & 1 \\ 2 & 2 & 10 \end{pmatrix} \tag{1.49}$$

Exercise 1.5.6. Now let us make a few observations about the two previous problems.

1. What will multiplying A by a matrix of the form

$$\begin{pmatrix} 1 & 0 & 0 \\ c & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \tag{1.50}$$

do?

2. What will multiplying A by a matrix of the form

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ c & 0 & 1 \end{pmatrix} \tag{1.51}$$

do?

3. What will multiplying A by a matrix of the form

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & c & 1 \end{pmatrix} \tag{1.52}$$

do?

4. More generally: If you wanted to multiply row j of an $n \times n$ matrix by c and add it to row k, that is the same as multiplying by what matrix?

Exercise 1.5.7. After doing all of the matrix products, $L_3 L_2 L_1 A$, the resulting matrix will have zeros in the entire lower triangle. That is, all of the non-zero entries of the resulting matrix will be on the main diagonal or above. We call this matrix U, for upper-triangular. Hence, we have formed a matrix

$$L_3 L_2 L_1 A = U \tag{1.53}$$

and if we want to solve for A we would get

$$A = (_____)^{-1}(_____)^{-1}(_____)^{-1}U \tag{1.54}$$

(Take care that everything is in the right order in your answer.)

Exercise 1.5.8. It would be nice, now, if the inverses of the L matrices were easy to find. Use `np.linalg.inv()` to directly compute the inverse of L_1, L_2, and L_3 for each of the example matrices. Then complete the statement: If L_k is an identity matrix with some non-zero c in row i and column j then L_k^{-1} is what matrix?

Exercise 1.5.9. We started this discussion with A as

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{pmatrix} \tag{1.55}$$

and we defined

$$L_1 = \begin{pmatrix} 1 & 0 & 0 \\ -4 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad L_2 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -7 & 0 & 1 \end{pmatrix}, \quad \text{and} \quad L_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -2 & 1 \end{pmatrix}. \tag{1.56}$$

Based on your answer to the previous exercises we know that

$$A = L_1^{-1} L_2^{-1} L_3^{-1} U. \tag{1.57}$$

Explicitly write down the matrices L_1^{-1}, L_2^{-1}, and L_3^{-1}.

Now explicitly find the product $L_1^{-1} L_2^{-1} L_3^{-1}$ and call this product L. Verify that L itself is also a lower-triangular matrix with ones on the main diagonal. Moreover, take note of exactly the form of the matrix. The answer should be super surprising to you!!

Throughout all of the preceding exercises, our final result is that we have factored the matrix A into the product of a lower-triangular matrix and an upper-triangular matrix. Stop and think about that for a minute … we just factored a matrix!

Let us return now to our discussion of solving the system of equations $Ax = b$. If A can be factored into $A = LU$ then the system of equations can be rewritten as $LUx = b$. As we will see in the next subsection, solving systems of equations with triangular matrices is super fast and relatively simple! Hence, we have partially achieved our modified goal of reducing the row reduction into some simpler case.[3]

It remains to implement the LU decomposition (also called the LU factorization) in Python.

Example 1.5.1 (The LU Factorization)**.** The following Python function takes a square matrix A and outputs the matrices L and U such that $A = LU$. The entire code is given to you. It will be up to you in the next exercise to pick apart every step of the function.

```
def myLU(A):
    n = A.shape[0] # get the dimension of the matrix A
    L = np.eye(n) # Build the identity part of L
    U = np.copy(A) # start the U matrix as a copy of A
    for j in range(0,n-1):
        for i in range(j+1,n):
            mult = U[i,j] / U[j,j]
            U[i, :] = U[i, :] - mult * U[j,:]
            L[i,j] = mult
    return L,U
```

---

[3]Take careful note here. We have actually just built a special case of the LU decomposition. Remember that in row reduction you are allowed to swap the order of the rows, but in our LU algorithm we do not have any row swaps. The version of LU with row swaps is called LU with partial pivoting. We will not built the full partial pivoting algorithm in this text but feel free to look it up. The wikipedia page is a decent place to start. What you will find is that there are indeed many different versions of the LU decomposition.

Exercise 1.5.10. Go to Example 1.5.1 and go through every iteration of every loop **by hand** starting with the matrix

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{pmatrix}. \tag{1.58}$$

Give details of what happens at every step of the algorithm. I will get you started.

- `n=3`, `L` starts as an identity matrix of the correct size, and `U` starts as a copy of `A`.
- Start the outer loop: `j=0`: (`j` is the counter for the column)
  - Start the inner loop: `i=1`: (`i` is the counter for the row)
    - `mult = A[1,0] / A[0,0]` so `mult`$= 4/1$.
    - `A[1, 1:3] = A[1, 1:3] - 4 * A[0,1:3]`. Translated, this states that columns 1 and 2 of matrix A took their original value minus 4 times the corresponding values in row 0.
    - `U[1, 1:3] = A[1, 1:3]`. Now we replace the locations in U with the updated information from our first step of row reduction.
    - `L[1,0]=4`. We now fill the L matrix with the proper value.
    - `U[1,0]=0`. Finally, we zero out the lower triangle piece of the U matrix which we have now taken care of.
  - `i=2`:
    - … keep going from here …

Exercise 1.5.11. Apply your new `myLU` code to other square matrices and verify that indeed A is the product of the resulting L and U matrices. You can produce a random matrix with `np.random.randn(n,n)` where `n` is the number of rows and columns of the matrix. For example, `np.random.randn(10,10)` will produce a random 10×10 matrix with entries chosen from the normal distribution with centre 0 and standard deviation 1. Random matrices are just as good as any other when testing your algorithm.

1.5.3 Solving Triangular Systems

We now know that row reduction is just a collection of sneaky matrix multiplications. In the previous exercises we saw that we can often turn our system of equations $Ax = b$ into the system $LUx = b$ where L is lower-triangular (with ones on the main diagonal) and U is upper-triangular. But why was this important?

Well, if $LUx = b$ then we can rewrite our system of equations as two systems:

$$\text{An upper-triangular system: } Ux = y \tag{1.59}$$

and

$$\text{A lower-triangular system: } Ly = b. \tag{1.60}$$

In the following exercises we will devise algorithms for solving triangular systems. After we know how to work with triangular systems we will put all of the pieces together and show how to leverage the LU decomposition and the solution techniques for triangular systems to quickly and efficiently solve linear systems.

Exercise 1.5.12. Outline a fast algorithm (without formal row reduction) for solving the lower-triangular system

$$\begin{pmatrix} 1 & 0 & 0 \\ 4 & 1 & 0 \\ 7 & 2 & 1 \end{pmatrix} \begin{pmatrix} y_0 \\ y_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 2 \end{pmatrix}. \tag{1.61}$$

Exercise 1.5.13. As a convention we will always write our lower-triangular matrices with ones on the main diagonal. Generalize your steps from the previous exercise so that you have an algorithm for solving any lower-triangular system. The most natural algorithm that most people devise here is called **forward substitution**.

Definition 1.5.1 (Forward Substutition Algorithm (`lsolve`))**.** The general statement of the Forward Substitution Algorithm is:

Solve $Ly = b$ for y, where the matrix L is assumed to be lower-triangular with ones on the main diagonal.

The code below gives a full implementation of the **Forward Substitution** algorithm (also called the `lsolve` algorithm).

```
def lsolve(L, b):
    # L is assumed to be a lower-triangular np.array
    n = b.size # what does this do?
    y = np.zeros(n) # what does this do?
    for i in range(n):
        # start the loop by assigning y to the value on the right
        y[i] = b[i]
        for j in range(i): # now adjust y
            y[i] = y[i] - L[i,j] * y[j]
    return(y)
```

Exercise 1.5.14. Work with your partner(s) to apply the `lsolve()` code to the lower-triangular system

$$\begin{pmatrix} 1 & 0 & 0 \\ 4 & 1 & 0 \\ 7 & 2 & 1 \end{pmatrix} \begin{pmatrix} y_0 \\ y_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 2 \end{pmatrix} \tag{1.62}$$

by hand. It is incredibly important to implement numerical linear algebra routines by hand a few times so that you truly understand how everything is being tracked and calculated.

I will get you started.

- Start: `i=0`:
  - `y[0]=1` since `b[0]=1`.
  - The next `for` loop does not start since `range(0)` has no elements (stop and think about why this is).
- Next step in the loop: `i=1`:
  - `y[1]` is initialized as 0 since `b[1]=0`.
  - Now we enter the inner loop at `j=0`:
    - What does `y[1]` become when `j=0`?
  - Does `j` increment to anything larger?
- Finally we increment `i` to `i=2`:
  - What does `y[2]` get initialized to?

  - Enter the inner loop at `j=0`:
    - What does `y[2]` become when `j=0`?
  - Increment the inner loop to `j=1`:
    - What does `y[2]` become when `j=1`?

- Stop

Exercise 1.5.15. Copy the code from Definition 1.5.1 into a Python function but in your code write a comment on every line stating what it is doing. Write a test script that creates a lower-triangular matrix of the correct form and a right-hand side b and solve for y. Test your code by giving it a large lower-triangular system.

Now that we have a method for solving lower-triangular systems, let us build a similar method for solving upper-triangular systems. The merging of lower and upper-triangular systems will play an important role in solving systems of equations.

Exercise 1.5.16. Outline a fast algorithm (without formal row reduction) for solving the upper-triangular system

$$\begin{pmatrix} 1 & 2 & 3 \\ 0 & -3 & -6 \\ 0 & 0 & -9 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ -4 \\ 3 \end{pmatrix} \tag{1.63}$$

The most natural algorithm that most people devise here is called **backward substitution**. Notice that in our upper-triangular matrix we do not have a diagonal containing all ones.

Exercise 1.5.17. Generalize your backward substitution algorithm from the previous problem so that it could be applied to any upper-triangular system.

Definition 1.5.2 (Backward Substitution Algorithm)**.** The following code solves the problem $Ux = y$ using backward substitution. The matrix U is assumed to be upper-triangular. You will notice that most of this code is incomplete. It is your job to complete this code, and the next exercise should help.

```
def usolve(U, y):
    # U is assumed to be an upper-triangular np.array
    n = y.size
    x = np.zeros(n)
    for i in range( ??? ):      # what should we be looping over?
        x[i] = y[i] / ???       # what should we be dividing by?
        for j in range( ??? ): # what should we be looping over:
            x[i] = x[i] - U[i,j] * x[j] / ??? # complete this line
            # ... what does the previous line do?
    return(x)
```

Exercise 1.5.18. Now we will work through the backward substitution algorithm to help fill in the blanks in the code. Consider the upper-triangular system

$$\begin{pmatrix} 1 & 2 & 3 \\ 0 & -3 & -6 \\ 0 & 0 & -9 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ -4 \\ 3 \end{pmatrix} \tag{1.64}$$

Work the code from Definition 1.5.2 to solve the system. Keep track of all of the indices as you work through the code. You may want to work this problem in conjunction with the previous two problems to unpack all of the parts of the *backward substitution* algorithm.

I will get you started.

- In your backward substitution algorithm you should have started with the last row, therefore the outer loop starts at `n-1` and reads backward to 0. (Why are we starting at `n-1` and not `n`?)
- Outer loop: `i=2`:
  - We want to solve the equation $-9x_2 = 3$ so the clear solution is to divide by -9. In code this means that `x[2]=y[2]/U[2,2]`.
  - There is nothing else to do for row 3 of the matrix, so we should not enter the inner loop. How can we keep from entering the inner loop?
- Outer loop: `i=1`:

- Now we are solving the algebraic equation $-3x_1 - 6x_2 = -4$. If we follow the high school algebra we see that $x_1 = \frac{-4-(-6)x_2}{-3}$ but this can be rearranged to

$$x_1 = \frac{-4}{-3} - \frac{-6x_2}{-3}. \tag{1.65}$$

  So we can initialize x_1 with $x_1 = \frac{-4}{-3}$. In code, this means that we initialize with `x[1] = y[1] / U[1,1]`.

- Now we need to enter the inner loop at `j=2`: (why are we entering the loop at `j=2`?)
  - To complete the algebra we need to take our initialized value of `x[1]` and subtract off $\frac{-6x_2}{-3}$. In code this is `x[1] = x[1] - U[1,2] * x[2] / U[1,1]`
- There is nothing else to do so the inner loop should end.

- Outer loop: `i=0`:
  - Finally, we are solving the algebraic equation $x_0 + 2x_1 + 3x_2 = 1$ for x_0. The clear and obvious solution is $x_0 = \frac{1-2x_1-3x_2}{1}$ (why am I explicitly showing the division by 1 here?).
  - Initialize x_0 at `x[0] = ???`
  - Enter the inner loop at `j=2`:
    - Adjust the value of `x[0]` by subtracting off $\frac{3x_2}{1}$. In code we have `x[0] = x[0] - ??? * ??? / ???`
  - Increment `j` to `j=1`:
    - Adjust the value of `x[0]` by subtracting off $\frac{2x_1}{1}$. In code we have `x[0] = x[0] - ??? * ??? / ???`
- Stop.
- You should now have a solution to the equation $Ux = y$. Substitute your solution in and verify that your solution is correct.

Exercise 1.5.19. Copy the code from Definition 1.5.2 into a Python function but in your code write a comment on every line stating what it is doing. Write a test script that creates an upper-triangular matrix of the correct form and a right-hand side y and solve for x. Your code needs to work on systems of arbitrarily large size.

1.5.4 Solving Systems with LU Decomposition

We are finally ready for the punch line of this whole LU and triangular systems business!

Exercise 1.5.20. If we want to solve $Ax = b$ then

1. If we can, write the system of equations as $LUx = b$.
2. Solve $Ly = b$ for y using forward substitution.
3. Solve $Ux = y$ for x using backward substitution.

Pick a matrix A and a right-hand side b and solve the system using this process.

Exercise 1.5.21. Try the process again on the 3×3 system of equations

$$\begin{pmatrix} 3 & 6 & 8 \\ 2 & 7 & -1 \\ 5 & 2 & 2 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} -13 \\ 4 \\ 1 \end{pmatrix} \tag{1.66}$$

That is: Find matrices L and U such that $Ax = b$ can be written as $LUx = b$. Then do two triangular solves to determine x.

Let us take stock of what we have done so far.

- Solving lower-triangular systems is super fast and easy!
- Solving upper-triangular systems is super fast and easy (so long as we never divide by zero).
- It is often possible to rewrite the matrix A as the product of a lower-triangular matrix L and an upper-triangular matrix U so $A = LU$.
- Now we can re-frame the equation $Ax = b$ as $LUx = b$.
- Substitute $y = Ux$ so the system becomes $Ly = b$. Solve for y with forward substitution.
- Now solve $Ux = y$ using backward substitution.

We have successfully take row reduction and turned into some fast matrix multiplications and then two very quick triangular solves. Ultimately this will be a faster algorithm for solving a system of linear equations.

Definition 1.5.3 (Solving Linear Systems with the LU Decomposition)**.** Let A be a square matrix in $\mathbb{R}^{n\times n}$ and let $x, b \in \mathbb{R}^n$. To solve the problem $Ax = b$,

1. Factor A into lower and upper-triangular matrices $A = LU$.
   `L, U = myLU(A)`
2. The system can now be written as $LUx = b$. Substitute $Ux = y$ and solve the system $Ly = b$ with forward substitution. `y = lsolve(L,b)`
3. Finally, solve the system $Ux = y$ with backward substitution.
   `x = usolve(U,y)`

Exercise 1.5.22. The LU decomposition is not perfect. Discuss where the algorithm will fail.

Exercise 1.5.23. What happens when you try to solve the system of equations

$$\begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 7 \\ 9 \\ -3 \end{pmatrix} \tag{1.67}$$

with the LU decomposition algorithm? Discuss.

1.6 The QR Factorization

In this section we will try to find an improvement on the LU factorization scheme from the previous section. What we will do here is leverage the geometry of the column space of the A matrix instead of leveraging the row reduction process.

Exercise 1.6.1. We want to solve the system of equations

$$\begin{pmatrix} 1/3 & 2/3 & 2/3 \\ 2/3 & 1/3 & -2/3 \\ -2/3 & 2/3 & -1/3 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 6 \\ 12 \\ -9 \end{pmatrix}. \tag{1.68}$$

1. We could do row reduction by hand … yuck … do not do this.
2. We could apply our new-found skills with the LU decomposition to solve the system, so go ahead and do that with your Python code.
3. What do you get if you compute the product $A^T A$?
   a. Why do you get what you get? In other words, what was special about A that gave such an nice result?
   b. What does this mean about the matrices A and A^T?
4. Now let us leverage what we found in part (3) to solve the system of equations $Ax = b$ much faster. Multiply both sides of the matrix equation by A^T, and now you should be able to just read off the solution. This seems amazing!!
5. What was it about this particular problem that made part (4) so elegant and easy?

The previous exercise tells us something amazing:

Theorem 1.6.1 (Orthonomal Matrices)**.** *If A is an orthonormal matrix where the columns are mutually orthogonal and every column is a unit vector, then $A^T = A^{-1}$ and to solve the system of equation $Ax = b$ we simply need to multiply both sides of the equation by A^T. Hence, the solution to $Ax = b$ is just $x = A^T b$ in this special case.*

Theorem 1.6.1 begs an obvious question: *Is there a way to turn any matrix A* into an orthogonal matrix so that we can solve $Ax = b$ in this same very efficient and fast way?

The answer: Yes. Kind of.

In essence, if we can factor our coefficient matrix into an orthonormal matrix and some other nicely formatted matrix (like a triangular matrix, perhaps) then the job of solving the linear system of equations comes down to matrix multiplication and a quick triangular solve – both of which are extremely fast!

What we will study in this section is a new matrix factorization called the QR factorization whose goal is to convert the matrix A into a product of two matrices, Q and R, where Q is orthonormal and R is upper-triangular.

Exercise 1.6.2. Let us say that we have a matrix A and we know that it can be factored into $A = QR$ where Q is an orthonormal matrix and R is an upper-triangular matrix. How would we then leverage this factorization to solve the system of equation $Ax = b$ for x?

Before proceeding to the algorithm for the QR factorization let us pause for a moment and review scalar and vector projections from Linear Algebra. In Figure 1.1 we see a graphical depiction of the vector u projected onto vector v. Notice that the projection is indeed the perpendicular projection as this is what seems natural geometrically.

The **vector projection** of u onto v is the vector cv. That is, the vector projection of u onto v is a scalar multiple of the vector v. The value of the scalar c is called the **scalar projection** of u onto v.

Figure 4.1: Projection of one vector onto another.

We can arrive at a formula for the scalar projection rather easily if we consider that the vector w in Figure 1.1 must be perpendicular to cv. Hence

$$w \cdot (cv) = 0. \tag{1.69}$$

From vector geometry we also know that $w = u - cv$. Therefore

$$(u - cv) \cdot (cv) = 0. \tag{1.70}$$

If we distribute we can see that

$$cu \cdot v - c^2 v \cdot v = 0 \tag{1.71}$$

and therefore either $c = 0$, which is only true if $u \perp v$, or

$$c = \frac{u \cdot v}{v \cdot v} = \frac{u \cdot v}{\|v\|^2}. \tag{1.72}$$

Therefore,

- the **scalar projection of** u onto v is

$$c = \frac{u \cdot v}{\|v\|^2} \tag{1.73}$$

- the **vector projection of** u onto v is

$$cv = \left(\frac{u \cdot v}{\|v\|^2}\right) v \tag{1.74}$$

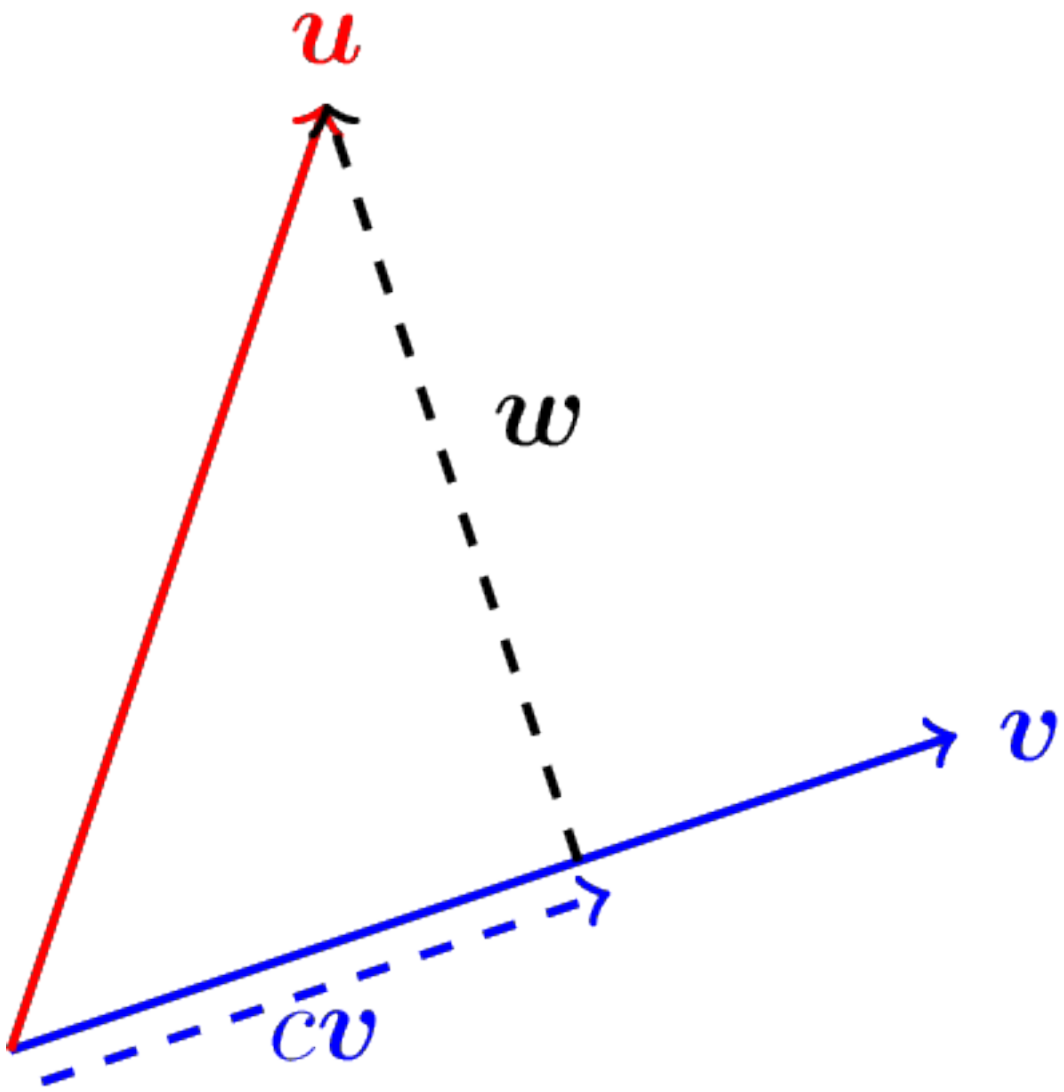

Figure 1.1: Projection of one vector onto another.

Another problem related to scalar and vector projections is to take a basis for the column space of a matrix and transform that basis into an orthogonal (or orthonormal) basis. Indeed, in Figure 1.1 if we have the matrix

$$A = \begin{pmatrix} | & | \\ u & v \\ | & | \end{pmatrix} \tag{1.75}$$

it should be clear from the picture that the columns of this matrix are not perpendicular. However, if we take the vector v and the vector w we do arrive at two orthogonal vector that form a basis for the same space. Moreover, if we normalize these vectors (by dividing by their respective lengths) then we can easily transform the original basis for the column space of A into an orthonormal basis. This process is called the Gram-Schmidt process, and you have encountered it in your Linear Algebra module.

Now we return to our goal of finding a way to factor a matrix A into an orthonormal matrix Q and an upper-triangular matrix R. The algorithm that we are about to build depends greatly on the ideas of scalar and vector projections.

Exercise 1.6.3. We want to build a QR factorization of the matrix A in the matrix equation $Ax = b$ so that we can leverage the fact that solving the equation $QRx = b$ is easy. Consider the matrix A defined as

$$A = \begin{pmatrix} 3 & 1 \\ 4 & 1 \end{pmatrix}. \tag{1.76}$$

Notice that the columns of A are NOT orthonormal (they are not unit vectors and they are not perpendicular to each other).

1. Draw a picture of the two column vectors of A in $\mathbb{R}^2$. we will use this picture to build geometric intuition for the rest of the QR factorization process.
2. Define a_0 as the first column of A and a_1 as the second column of A. That is

$$a_0 = \begin{pmatrix} 3 \\ 4 \end{pmatrix} \quad \text{and} \quad a_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}. \tag{1.77}$$

   Turn a_0 into a unit vector and call this unit vector q_0

$$q_0 = \frac{a_0}{\|a_0\|} = \begin{pmatrix} \underline{\qquad} \\ \underline{\qquad} \end{pmatrix}. \tag{1.78}$$

   This vector q_0 will be the first column of the 2×2 matrix Q. Why is this a nice place to start building the Q matrix (think about the desired structure of Q)?

3. In your picture of a_0 and a_1 mark where q_0 is. Then draw the orthogonal projection from a_1 onto q_0. In your picture you should now see a right triangle with a_1 on the hypotenuse, the projection of a_1 onto q_0 on one leg, and the second leg is the vector difference of the hypotenuse and the first leg. Simplify the projection formula for leg 1 and write the formula for leg 2.

$$\text{hypotenuse } = a_1 \tag{1.79}$$

$$\text{leg 1 } = \left(\frac{a_1 \cdot q_0}{q_0 \cdot q_0}\right) q_0 = \underline{\qquad\qquad} \tag{1.80}$$

$$\text{leg 2 } = \underline{\qquad\qquad} - \underline{\qquad\qquad}. \tag{1.81}$$

4. Compute the vector for leg 2 and then normalize it to turn it into a unit vector. Call this vector q_1 and put it in the second column of Q.

5. Verify that the columns of Q are now orthogonal and are both unit vectors.

6. The matrix R is supposed to complete the matrix factorization $A = QR$. We have built Q as an orthonormal matrix. How can we use this fact to solve for the matrix R?

7. You should now have an orthonormal matrix Q and an upper-triangular matrix R. Verify that $A = QR$.

8. An alternate way to build the R matrix is to observe that

$$R = \begin{pmatrix} a_0 \cdot q_0 & a_1 \cdot q_0 \\ 0 & a_1 \cdot q_1 \end{pmatrix}. \tag{1.82}$$

   Show that this is indeed true for the matrix A from this problem.

Exercise 1.6.4. Keeping track of all of the arithmetic in the QR factorization process is quite challenging, so let us leverage Python to do some of the work for us. The following block of code walks through the previous exercise without any looping (that way we can see every step transparently). Some of the code is missing so you will need to fill it in.

```
import numpy as np
# Define the matrix $A$
A = np.array([[3,1],[4,1]])
n = A.shape[0]
# Build the vectors a0 and a1
a0 = A[??? , ???] # ... write code to get column 0 from A
a1 = A[??? , ???] # ... write code to get column 1 from A
# Set up storage for Q
Q = np.zeros( (n,n) )
# build the vector q0 by normalizing a0
q0 = a0 / np.linalg.norm(a0)
# Put q0 as the first column of Q
Q[:,0] = q0
# Calculate the lengths of the two legs of the triangle
leg1 = # write code to get the vector for leg 1 of the triangle
leg2 = # write code to get the vector for leg 2 of the triangle
# normalize leg2 and call it q1
q1 = # write code to normalize leg2
q1 = q1 / np.linalg.norm(q1) # Just to be safe with normalization if not implied
Q[:,1] = q1 # What does this line do?
R = # ... build the R matrix out of A and Q

print("The Q matrix is \n",Q,"\n")
print("The R matrix is \n",R,"\n")
print("The A matrix is \n",A,"\n")
print("The product QR is\n",Q @ R)
```

Exercise 1.6.5. You should notice that the code in the previous exercise does not depend on the specific matrix A that we used? Put in a different 2×2 matrix and verify that the process still works. That is, verify that Q is orthonormal, R is upper-triangular, and $A = QR$. Be sure, however, that your matrix A is full rank.

Exercise 1.6.6. Draw two generic vectors in $\mathbb{R}^2$ and demonstrate the process outlined in the previous problem to build the vectors for the Q matrix starting from your generic vectors.

Exercise 1.6.7. Now we will extend the process from the previous exercises to three dimensions. This time we will seek a matrix Q that has three orthonormal vectors starting from the three original columns of a 3×3 matrix A. Perform each of the following steps **by hand** on the matrix

$$A = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix}. \tag{1.83}$$

In the end you should end up with an orthonormal matrix Q and an upper-triangular matrix R.

- **Step 1**: Pick column a_0 from the matrix A and normalize it. Call this new vector q_0 and make that the first column of the matrix Q.
- **Step 2**: Project column a_1 of A onto q_0. This forms a right triangle with a_1 as the hypotenuse, the projection of a_1 onto q_0 as one of the legs, and the vector difference between these two as the second leg. Notice that the second leg of the newly formed right triangle is perpendicular to q_0 by design. If we normalize this vector then we have the second column of Q, q_1.
- **Step 3:** Now we need a vector that is perpendicular to both q_0 AND q_1. To achieve this we are going to project column a_2 from A onto the plane formed by q_0 and q_1. we will do this in two steps:
  - **Step 3a**: We first project a_2 down onto both q_0 and q_1.
  - **Step 3b**: The vector that is perpendicular to both q_0 and q_1 will be the difference between a_2 the projection of a_2 onto q_0 and the projection of a_2 onto q_1. That is, we form the vector $w = a_2 - (a_2 \cdot q_0)q_0 - (a_2 \cdot q_1)q_1$. Normalizing this vector will give us q_2. (Stop now and prove that q_2 is indeed perpendicular to both q_1 and q_0.)

The result should be the matrix Q which contains orthonormal columns. To build the matrix R we simply recall that $A = QR$ and $Q^{-1} = Q^T$ so $R = Q^T A$.

Exercise 1.6.8. Repeat the previous exercise but write code for each step so that Python can handle all of the computations. Again use the matrix

$$A = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix}. \tag{1.84}$$

Example 1.6.1. (**QR for** $n = 3$) For the sake of clarity let us now write down the full QR factorization for a 3×3 matrix.

If the columns of A are a_0, a_1, and a_2 then

$$q_0 = \frac{a_0}{\|a_0\|} \tag{1.85}$$

$$q_1 = \frac{a_1 - (a_1 \cdot q_0)\, q_0}{\|a_1 - (a_1 \cdot q_0)\, q_0\|} \tag{1.86}$$

$$q_2 = \frac{a_2 - (a_2 \cdot q_0)\, q_0 - (a_2 \cdot q_1)\, q_1}{\|a_2 - (a_2 \cdot q_0)\, q_0 - (a_2 \cdot q_1)\, q_1\|} \tag{1.87}$$

and

$$R = \begin{pmatrix} a_0 \cdot q_0 & a_1 \cdot q_0 & a_2 \cdot q_0 \\ 0 & a_1 \cdot q_1 & a_2 \cdot q_1 \\ 0 & 0 & a_2 \cdot q_2 \end{pmatrix} \tag{1.88}$$

Exercise 1.6.9 (The QR Factorization)**.** Now we are ready to build general code for the QR factorization. The following Python function definition is partially complete. Fill in the missing pieces of code and then test your code on square matrices of many different sizes. The easiest way to check if you have an error is to find the normed difference between A and QR with `np.linalg.norm(A - Q*R)`.

```
import numpy as np
def myQR(A):
    n = A.shape[0]
    Q = np.zeros( (n,n) )
    for j in range( ??? ): # The outer loop goes over the columns
        q = A[:,j]
        # The next loop is meant to do all of the projections.
        # When do you start the inner loop and how far do you go?
        # Hint: You do not need to enter this loop the first time
        for i in range( ??? ):
            length_of_leg = np.dot(A[:,j], Q[:,i])
            q = q -  ??? * ??? # This is where we do projections
        Q[:,j] = q / np.linalg.norm(q)
    R = # finally build the R matrix
    return Q, R
```

```
# Test Code
A = np.array( ... )
# or you can build A with use np.random.randn()
# Often time random matrices are good test cases
Q, R = myQR(A)
error = np.linalg.norm(A - Q @ R)
print(error)
```

We now have a robust algorithm for doing QR factorization of square matrices we can finally return to solving systems of equations.

Theorem 1.6.2. *(**Solving Systems with** QR) Remember that we want to solve $Ax = b$ and since $A = QR$ we can rewrite it with $QRx = b$. Since we know that Q is orthonormal by design we can multiply both sides of the equation by Q^T to get $Rx = Q^T b$. Finally, since R is upper-triangular we can use our* `usolve` *code from the previous section to solve the resulting triangular system.*

Exercise 1.6.10. Solve the system of equations

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 2 \end{pmatrix} \tag{1.89}$$

by first computing the QR factorization of A and then solving the resulting upper-triangular system.

Exercise 1.6.11. Write code that builds a random $n \times n$ matrix and a random $n \times 1$ vector. Solve the equation $Ax = b$ using the QR factorization and compare the answer to what we find from `np.linalg.solve()`. Do this many times for various values of n and create a plot with n on the horizontal axis and the normed error between Python's answer and your answer from the QR algorithm on the vertical axis. It would be wise to use a `plt.semilogy()` plot. To find the normed difference you should use `np.linalg.norm()`. What do you notice?

1.7 Over-determined Systems and Curve Fitting

Exercise 1.7.1. Consider the problem of finding the quadratic function $f(x) = ax^2 + bx + c$ that *best fits* the points

$$(0, 1.07), (1, 3.9), (2, 14.8), (3, 26.8). \tag{1.90}$$

We do not know the values of a, b, or c but we do have four different (x, y) ordered pairs. Hence, we have four equations:

$$1.07 = a(0)^2 + b(0) + c \tag{1.91}$$

$$3.9 = a(1)^2 + b(1) + c \tag{1.92}$$

$$14.8 = a(2)^2 + b(2) + c \tag{1.93}$$

$$26.8 = a(3)^2 + b(3) + c. \tag{1.94}$$

There are four equations and only three unknowns. This is what is called an **over determined systems** – when there are more equations than unknowns. Let us play with this problem.

1. First turn the system of equations into a matrix equation.

$$\begin{pmatrix} 0 & 0 & 1 \\ __ & __ & __ \\ __ & __ & __ \\ __ & __ & __ \end{pmatrix} \begin{pmatrix} a \\ b \\ c \end{pmatrix} = \begin{pmatrix} 1.07 \\ 3.9 \\ 14.8 \\ 26.8 \end{pmatrix}. \tag{1.95}$$

2. None of our techniques for solving systems will likely work here since it is highly unlikely that the vector on the right-hand side of the equation is in the column space of the coefficient matrix. Discuss this.

3. One solution to the unfortunate fact from part (b) is that we can project the vector on the right-hand side into the subspace spanned by the columns of the coefficient matrix. Think of this as casting the shadow of the right-hand vector down onto the space spanned by the columns. If we do this projection we will be able to solve the equation for the values of a, b, and c that will create the projection exactly – and hence be as close as we can get to the actual right-hand side. Draw a picture of what we have said here.

4. Now we need to project the right-hand side, call it b, onto the column space of the the coefficient matrix A. Recall the following facts:

- Projections are dot products
- Matrix multiplication is nothing but a bunch of dot products.
- The projections of b onto the columns of A are the dot products of b with each of the columns of A.
- What matrix can we multiply both sides of the equation $Ax = b$ by in order for the right-hand side to become the projection that we want? (Now do the projection in Python)

5. If you have done part (d) correctly then you should now have a square system (i.e. the matrix on the left-hand side should now be square). Solve this system for a, b, and c.

Theorem 1.7.1 (Solving Overdetermined Systems)**.** *If $Ax = b$ is an overdetermined system (i.e. A has more rows than columns) then we first multiply both sides of the equation by A^T (why do we do this?) and then solve the square system of equations $(A^T A)x = A^T b$ using a system solving like LU or QR. The answer to this new system is interpreted as the vector x which solves exactly for the projection of b onto the column space of A.*

The equation $(A^T A)x = A^T b$ is called ***the normal equations*** *and arises often in Statistics and Machine Learning.*

Exercise 1.7.2. Fit a linear function to the following data. Solve for the slope and intercept using the technique outlined in Theorem 1.7.1. Make a plot of the points along with your best fit curve.

x	y
0	4.6
1	11
2	12
3	19.1
4	18.8
5	39.5
6	31.1
7	43.4
8	40.3
9	41.5

x	y
10	41.6

Exercise 1.7.3. Fit a quadratic function to the following data using the technique outlined in Theorem 1.7.1. Make a plot of the points along with your best fit curve.

x	y
0	-6.8
1	11.8
2	50.6
3	94
4	224.3
5	301.7
6	499.2
7	454.7
8	578.5
9	1102
10	1203.2

Code to download the data directly is given below.

```
import numpy as np
import pandas as pd
URL1 = 'https://raw.githubusercontent.com/NumericalMethodsSullivan'
URL2 = '/NumericalMethodsSullivan.github.io/master/data/'
URL = URL1+URL2
data = np.array( pd.read_csv(URL+'Exercise4_52.csv') )
# Exercise4_52.csv
```

Exercise 1.7.4. The Statistical technique of curve fitting is often called "linear regression." This even holds when we are fitting quadratic functions, cubic functions, etc to the data … we still call that linear regression! Why?

This section of the text on solving over determined systems is just a bit of a teaser for a bit of higher-level statistics, data science, and machine learning. The normal equations and solving systems via projections is the starting point of many modern machine learning algorithms.

1.8 The Eigenvalue-Eigenvector Problem

We finally turn our attention to another major topic in numerical linear algebra.[4]

Definition 1.8.1 (The Eigenvalue Problem)**.** Recall that the eigenvectors, x, and the eigenvalues, λ of a square matrix satisfy the equation $Ax = \lambda x$. Geometrically, the eigen-problem is the task of finding the special vectors x such that multiplication by the matrix A only produces a scalar multiple of x.

Thinking about matrix multiplication, the geometric notion of the eigenvalue problem is rather peculiar since matrix-vector multiplication usually results in a scaling and a rotation of the vector x. Therefore, in some sense the eigenvectors are the only special vectors which avoid geometric rotation under matrix multiplication. For a graphical exploration of this idea see:
https://www.geogebra.org/m/JP2XZpzV.

Recall that to solve the eigen-problem for a square matrix A we complete the following steps:

1. First rearrange the definition of the eigenvalue-eigenvector pair to

$$(Ax - \lambda x) = 0. \tag{1.96}$$

2. Next, factor the x on the right to get

$$(A - \lambda I)x = 0. \tag{1.97}$$

[4]Numerical Linear Algebra is a huge field and there is way more to say … but alas, this is an introductory course in Numerical Analysis so we cannot do everything. Sigh.

3. Now observe that since $x \neq 0$ the matrix $A - \lambda I$ must NOT have an inverse. Therefore,
$$\det(A - \lambda I) = 0. \tag{1.98}$$

4. Solve the equation $\det(A - \lambda I) = 0$ for all of the values of λ.

5. For each λ, find a solution to the equation $(A - \lambda I)x = 0$. Note that there will be infinitely many solutions so you will need to make wise choices for the free variables.

Exercise 1.8.1. Find the eigenvalues and eigenvectors of
$$A = \begin{pmatrix} 1 & 2 \\ 4 & 3 \end{pmatrix}. \tag{1.99}$$

Exercise 1.8.2. In the matrix
$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} \tag{1.100}$$
one of the eigenvalues is $\lambda_1 = 0$.

1. What does that tell us about the matrix A?

2. What is the eigenvector v_1 associated with $\lambda_1 = 0$?

3. What is the null space of the matrix A?

OK. Now that you have recalled some of the basics, let us play with a little limit problem. The following exercises are going to work us toward the **power method** for finding certain eigen-structures of a matrix.

Exercise 1.8.3. Consider the matrix

$$A = \begin{pmatrix} 8 & 5 & -6 \\ -12 & -9 & 12 \\ -3 & -3 & 5 \end{pmatrix}. \tag{1.101}$$

This matrix has the following eigen-structure:

$$v_1 = \begin{pmatrix} 1 \\ -1 \\ 0 \end{pmatrix} \quad \text{with} \quad \lambda_1 = 3 \tag{1.102}$$

$$v_2 = \begin{pmatrix} 2 \\ 0 \\ 2 \end{pmatrix} \quad \text{with} \quad \lambda_2 = 2 \tag{1.103}$$

$$v_3 = \begin{pmatrix} -1 \\ 3 \\ 1 \end{pmatrix} \quad \text{with} \quad \lambda_3 = -1 \tag{1.104}$$

If we have

$$x = -2v_1 + 1v_2 - 3v_3 = \begin{pmatrix} 3 \\ -7 \\ -1 \end{pmatrix} \tag{1.105}$$

then we want to do a bit of an experiment. What happens when we iteratively multiply x by A but at the same time divide by the largest eigenvalue. Let us see:

- What is $A^1x/3^1$?
- What is $A^2x/3^2$?
- What is $A^3x/3^3$?
- What is $A^4x/3^4$?
- ...

It might be nice now to go to some Python code to do the computations (if you have not already). Use your code to conjecture about the following limit.

$$\lim_{k\to\infty} \frac{A^k x}{\lambda_{max}^k} = ???. \tag{1.106}$$

In this limit we are really interested in the direction of the resulting vector, not the magnitude. Therefore, in the code below you will see that we normalize the resulting vector so that it is a unit vector.

Note: be careful, computers do not do infinity, so for powers that are too large you will not get any results.

```
import numpy as np
A = np.array([[8,5,-6],[-12,-9,12],[-3,-3,5]])
x = np.array([[3],[-7],[-1]])
eigval_max = 3

k = 4
# Note: For numpy arrays, A**k is element-wise power.
# We must use np.linalg.matrix_power for matrix power.
result = np.linalg.matrix_power(A, k) @ x / eigval_max**k
print(result / np.linalg.norm(result) )
```

Exercise 1.8.4. If a matrix A has eigenvectors v_1, v_2, v_3, $\cdots$, v_n with eigenvalues $\lambda_1, \lambda_2, \lambda_3, \dots, \lambda_n$ and x is in the column space of A then what will we get, approximately, if we evaluate $A^k x / \max_j(\lambda_j)^k$ for very large values of k?

Discuss your conjecture with your peers. Then try to verify it with several numerical examples.

Exercise 1.8.5. Explain your result from the previous exercise geometrically.

Exercise 1.8.6. The algorithm that we have been toying with will find the dominant eigenvector of a matrix fairly quickly. Why might you be only interested in the dominant eigenvector of a matrix? Discuss.

Exercise 1.8.7. In this problem we will formally prove the conjecture that you just made. This conjecture will lead us to the **power method** for finding the dominant eigenvector and eigenvalue of a matrix.

1. Assume that A has n linearly independent eigenvectors $v_1, v_2, \dots, v_n$ and choose $x = \sum_{j=1}^{n} c_j v_j$. You have proved in the past that

$$A^k x = c_1 \lambda_1^k v_1 + c_2 \lambda_2^k v_2 + \cdots c_n \lambda_n^k v_n. \tag{1.107}$$

   Stop and sketch out the details of this proof now.

2. If we factor λ_1^k out of the right-hand side we get

$$A^k x = \lambda_1^k \left(c_1 ??? + c_2 \left(\frac{???}{???}\right)^k v_2 + c_3 \left(\frac{???}{???}\right)^k v_3 + \cdots + c_n \left(\frac{???}{???}\right)^k v_n \right) \quad (1.108)$$

   (fill in the question marks)

3. If $|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \cdots \geq |\lambda_n|$ then what happens to each of the $(\lambda_j/\lambda_1)^k$ terms as $k \to \infty$?

4. Using your answer to part (c), what is $\lim_{k\to\infty} A^k x/\lambda_1^k$?

Theorem 1.8.1 (The Power Method)**.** *The following algorithm, called* ***the power method*** *will quickly find the eigenvalue of largest absolute value for a square matrix $A \in \mathbb{R}^{n\times n}$ as well as the associated (normalized) eigenvector. We are assuming that there are n linearly independent eigenvectors of A.*

Step 1: *Given a non-zero vector x, set $v^{(1)} = x/\|x\|$. (Here the superscript indicates the iteration number) Note that the initial vector x is pretty irrelevant to the process so it can just be a random vector of the correct size..*

Step 2: *For $k = 2, 3, ...$*

Step 2a: *Compute $\tilde{v}^{(k)} = Av^{(k-1)}$ (this gives a non-normalized version of the next estimate of the dominant eigenvector.)*

Step 2b: *Set $\lambda^{(k)} = \tilde{v}^{(k)} \cdot v^{(k-1)}$. (this gives an approximation of the eigenvalue since if $v^{(k-1)}$ was the actual eigenvector we would have $\lambda = Av^{(k-1)} \cdot v^{(k-1)}$. Stop now and explain this.)*

Step 2c: *Normalize $\tilde{v}^{(k)}$ by computing $v^{(k)} = \tilde{v}^{(k)}/\|\tilde{v}^{(k)}\|$. (This guarantees that you will be sending a unit vector into the next iteration of the loop)*

Exercise 1.8.8. Go through Theorem 1.8.1 carefully and describe what we need to do in each step and why we are doing it. Then complete all of the missing pieces of the following Python function.

```
import numpy as np
def myPower(A, tol = 1e-8):
    n = A.shape[0]
    x = np.random.randn(n)
    x = # turn x into a unit vector
    # we do not actually need to keep track of the old iterates
    L = 1 # initialize the dominant eigenvalue
    counter = 0 # keep track of how many steps we have taken
    # You can build a stopping rule from the definition
    # Ax = lambda x ...
    while (???) > tol and counter < 10000:
        x = A @ x # update the dominant eigenvector
        x = ??? # normalize
        L = ??? # approximate the eignevalue
        counter += 1 # increment the counter
    return x, L
```

Exercise 1.8.9. Test your `myPower()` function on several matrices where you know the eigenstructure. Then try the `myPower()` function on larger random matrices. You can check that it is working using `np.linalg.eig()` (be sure to normalize the vectors in the same way so you can compare them.)

Exercise 1.8.10. In the Power Method iteration you may end up getting a different sign on your eigenvector as compared to `np.linalg.eig()`. Why might this happen? Generate a few examples so you can see this. You can avoid this issue if you use a `while` loop in your Power Method code and the logical check takes advantage of the fact that we are trying to solve the equation $Ax = \lambda x$. Hint: $Ax = \lambda x$ is equivalent to $Ax - \lambda x = 0$.

Exercise 1.8.11. What happens in the power method iterations when λ_1 is complex. The maximum eigenvalue can certainly be complex if $|\lambda_1|$ (the modulus of the complex number) is larger than all of the other eigenvalues. It may be helpful to build a matrix specifically with complex eigenvalues.[5]

[5] To build a matrix with specific eigenvalues it may be helpful to recall the matrix factorization $A = PDP^{-1}$ where the columns of P are the eigenvectors of A and the diagonal entries of D are the eigenvalues. If you choose P and D then you can build A with your specific eigen-structure. If you are looking for complex eigenvalues then remember that the eigenvectors may well be complex too.

Exercise 1.8.12 (Convergence Rate of the Power Method)**.** The proof that the power method will work hinges on the fact that $|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \cdots \geq |\lambda_n|$. In Exercise 1.8.7 we proved that the limit

$$\lim_{k\to\infty} \frac{A^k x}{\lambda_1^k} \tag{1.109}$$

converges to the dominant eigenvector, but how fast is the convergence? What does the speed of the convergence depend on?

Take note that since we are assuming that the eigenvalues are ordered, the ratio λ_2/λ_1 will be larger than λ_j/λ_1 for all $j > 2$. Hence, the speed at which the power method converges depends mostly on the ratio λ_2/λ_1. Let us build a numerical experiment to see how sensitive the power method is to this ratio.

Build a 4×4 matrix A with dominant eigenvalue $\lambda_1 = 1$ and all other eigenvalues less than 1 in absolute value. Then choose several values of λ_2 and build an experiment to determine the number of iterations that it takes for the power method to converge to within a pre-determined tolerance to the dominant eigenvector. In the end you should produce a plot with the ratio λ_2/λ_1 on the horizontal axis and the number of iterations to converge to a fixed tolerance on the vertical axis. Discuss what you see in your plot.

Hint: To build a matrix with specific eigen-structure use the matrix factorization $A = PDP^{-1}$ where the columns of P contain the eigenvectors of A and the diagonal of D contains the eigenvalues. In this case the P matrix can be random but you need to control the D matrix. Moreover, remember that λ_3 and λ_4 should be smaller than λ_2.

1.9 Algorithm Summaries

Exercise 1.9.1. Explain in clear language how to efficiently solve an upper-triangular system of linear equations.

Exercise 1.9.2. Explain in clear language how to efficiently solve a lower-triangular system of linear equations.

Exercise 1.9.3. Explain in clear language how to solve the equation $Ax = b$ using an LU decomposition.

Exercise 1.9.4. Explain in clear language how to solve an overdetermined system of linear equations (more equations than unknowns) numerically.

Exercise 1.9.5. Explain in clear language the algorithm for finding the columns of the Q matrix in the QR factorization. Give all of the mathematical details.

Exercise 1.9.6. Explain in clear language how to find the upper-triangular matrix R in the QR factorization. Give all of the mathematical details.

Exercise 1.9.7. Explain in clear language how to solve the equation $Ax = b$ using a QR decomposition.

Exercise 1.9.8. Explain in clear language how the power method works to find the dominant eigenvalue and eigenvector of a square matrix. Give all of the mathematical details.

1.10 Problems

Exercise 1.10.1. As mentioned much earlier in this chapter, there is an `rref()` command in Python, but it is in the `sympy` library instead of the `numpy` library – it is implemented as a symbolic computation instead of a numerical computation. OK. So what? In this problem we want to compare the time to solve a system of equations $Ax = b$ with each of the following techniques:

- row reduction of an augmented matrix $\begin{pmatrix} A & | & b \end{pmatrix}$ with `sympy`,
- our implementation of the LU decomposition,

- our implementation of the QR decomposition, and
- the `numpy.linalg.solve()` command.

To time code in Python first import the `time` library. Then use `start = time.time()` at the start of your code and `stop = time.time()` and the end of your code. The difference between `stop` and `start` is the elapsed computation time.

Make observations about how the algorithms perform for different sized matrices. You can use random matrices and vectors for A and b. The end result should be a plot showing how the average computation time for each algorithm behaves as a function of the size of the coefficient matrix.

The code below will compute the reduced row echelon form of a matrix (RREF). Implement the code so that you know how it works.

```
import sympy as sp
import numpy as np
# in this problem it will be easiest to start with numpy matrices
A = np.array([[1, 0, 1], [2, 3, 5], [-1, -3, -3]])
b = np.array([[3],[7],[3]])
Augmented = np.c_[A,b] # augment b onto the right hand side of A

Msymbolic = sp.Matrix(Augmented)
MsymbolicRREF = Msymbolic.rref()
print(MsymbolicRREF)
```

To time code you can use code like the following.

```
import time
start = time.time()
# some code that you want to time
stop =  time.time()
total_time = stop - start
print("Total computation time=",total_time)
```

Exercise 1.10.2. Imagine that we have a 1 meter long thin metal rod that has been heated to 100° on the left-hand side and cooled to 0° on the right-hand side. We want to know the temperature every 10 cm from left to right on the rod.

1. First we break the rod into equal 10cm increments as shown. See Figure 1.2. How many unknowns are there in this picture?

2. The temperature at each point along the rod is the average of the temperatures at the adjacent points. For example, if we let T_1 be the temperature at point x_1 then

$$T_1 = \frac{T_0 + T_2}{2}. \tag{1.110}$$

Write a system of equations for each of the unknown temperatures.

3. Solve the system for the temperature at each unknown node using either LU or QR decomposition.

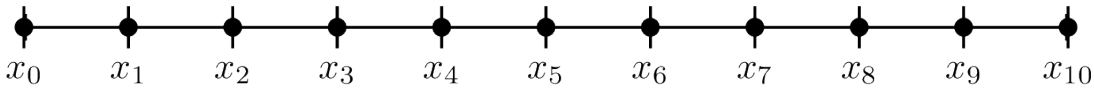

Figure 1.2: A rod to be heated broken into 10 equal-length segments.

Exercise 1.10.3. Write code to solve the following systems of equations via both LU and QR decompositions. If the algorithm fails then be sure to explain exactly why.

1.
$$\begin{aligned} x + 2y + 3z &= 4 \\ 2x + 4y + 3z &= 5 \\ x + y &= 4 \end{aligned} \tag{1.111}$$

2.
$$\begin{aligned} 2y + 3z &= 4 \\ 2x + 3z &= 5 \\ y &= 4 \end{aligned} \tag{1.112}$$

3.
$$\begin{aligned} 2y + 3z &= 4 \\ 2x + 4y + 3z &= 5 \\ x + y &= 4 \end{aligned} \tag{1.113}$$

Exercise 1.10.4. Give a specific example of a non-zero matrix which will NOT have an LU decomposition. Give specific reasons why LU will fail on your matrix.

Exercise 1.10.5. Give a specific example of a non-zero matrix which will NOT have an QR decomposition. Give specific reasons why QR will fail on your matrix.

Exercise 1.10.6. Have you ever wondered how scientific software computes a determinant? The formula that you learned for calculating determinants by hand is horribly cumbersome and computationally intractable for large matrices. This problem is meant to give you glimpse of what is *actually* going on under the hood.[6]

If A has an LU decomposition then $A = LU$. Use properties that you know about determinants to come up with a simple way to find the determinant for matrices that have an LU decomposition. Show all of your work in developing your formula.

Once you have your formula for calculating $\det(A)$, write a Python function that accepts a matrix, produces the LU decomposition, and returns the determinant of A. Check your work against Python's `np.linalg.det()` function.

Exercise 1.10.7. For this problem we are going to run a numerical experiment to see how the process of solving the equation $Ax = b$ using the LU factorization performs on random coefficient matrices A and random right-hand sides b. We will compare against Python's algorithm for solving linear systems.

We will do the following:

Create a loop that does the following:

1. Loop over the size of the matrix n.
2. Build a random matrix A of size $n \times n$. You can do this with the code `A = np.random.randn(n,n)`
3. Build a random vector b in $\mathbb{R}^n$. You can do this with the code `b = np.random.randn(n)`
4. Find Python's answer to the problem $Ax = b$ =0 using the command `exact = np.linalg.solve(A,b)`
5. Write code that uses your three LU functions (`myLU`, `lsolve`, `usolve`) to find a solution to the equation $Ax = b$.
6. Find the error between your answer and the exact answer using the code `np.linalg.norm(x - exact)`

[6] Actually, the determinant computation uses LU with partial pivoting which we did not cover here in the text. What we are looking at in this exercise is a smaller sub-case of what happens when you have a matrix A that does not require any row swaps in the row reduction process.

7. Make a plot (`plt.semilogy()`) that shows how the error behaves as the size of the problem changes. You should run this for matrices of larger and larger size but be warned that the loop will run for quite a long time if you go above 300×300 matrices. Just be patient.

Conclusions: What do you notice in your final plot. What does this tell you about the behaviour of our LU decomposition code?

Exercise 1.10.8. Repeat Exercise 1.10.7 for the QR decomposition. Your final plot should show both the behaviour of QR and of LU throughout the experiment. What do you notice?

Exercise 1.10.9. Find a least squares solution to the equation $Ax = b$ in two different ways with

$$A = \begin{pmatrix} 1 & 3 & 5 \\ 4 & -2 & 6 \\ 4 & 7 & 8 \\ 3 & 7 & 19 \end{pmatrix} \quad \text{and} \quad b = \begin{pmatrix} 5 \\ 2 \\ -2 \\ 8 \end{pmatrix}. \tag{1.114}$$

Exercise 1.10.10. Let A be defined as

$$A = \begin{pmatrix} 10^{-20} & 1 \\ 1 & 1 \end{pmatrix} \tag{1.115}$$

and let b be the vector

$$b = \begin{pmatrix} 2 \\ 3 \end{pmatrix}. \tag{1.116}$$

Notice that A has a tiny, but non-zero, value in the first entry.

1. Solve the linear system $Ax = b$ by hand.
2. Use your `myLU`, `lsolve`, and `usolve` functions to solve this problem using the LU decomposition method.
3. Compare your answers to parts (a) and (b). What went wrong?

Exercise 1.10.11 (Hilbert Matrices)**.** A Hilbert Matrix is a matrix of the form $H_{ij} = 1/(i+j+1)$ where both i and j both start indexed at 0. For example, a 4×4 Hilbert Matrix is

$$H = \begin{pmatrix} 1 & \frac{1}{2} & \frac{1}{3} & \frac{1}{4} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & \frac{1}{6} \\ \frac{1}{4} & \frac{1}{5} & \frac{1}{6} & \frac{1}{7} \end{pmatrix}. \tag{1.117}$$

This type of matrix is often used to test numerical linear algebra algorithms since it is known to have some *odd* behaviours … which you will see in a moment.

1. Write code to build a $n \times n$ Hilbert Matrix and call this matrix H. Test your code for various values of n to be sure that it is building the correct matrices.

2. Build a vector of ones called b with code `b = np.ones((n,1))`. We will use b as the right hand side of the system of equations $Hx = b$.

3. Solve the system of equations $Hx = b$ using any technique you like from this chapter.

4. Now let us say that you change the first entry of b by just a little bit, say 10^{-15}. If we were to now solve the equation $Hx_{new} = b_{new}$ what would you expect as compared to solving $Hx = b$.

5. Now let us actually make the change suggested in part (d). Use the code `bnew = np.ones((n,1))` and then `bnew[0] = bnew[0] + 1e-15` to build a new b vector with this small change. Solve $Hx = b$ and $Hx_{new} = b_{new}$ and then compare the maximum absolute difference `np.max(np.abs(x - xnew))`. What do you notice? Make a plot with n on the horizontal axis and the maximum absolute difference on the vertical axis. What does this plot tell you about the solution to the equation $Hx = b$?

6. We know that HH^{-1} should be the identity matrix. As we will see, however, Hilbert matrices are particularly poorly behaved! Write a loop over n that (i) builds a Hilbert matrix of size n, (ii) calculates HH^{-1} (using `np.linalg.inv()` to compute the inverse directly), (iii) calculates the norm of the difference between the identity matrix (`np.identity(n)`) and your calculated identity matrix from part (ii). Finally. Build a plot that shows n on the horizontal axis and the normed difference on the vertical axis. What do you see? What does this mean about the matrix inversion of the Hilbert matrix.

7. There are cautionary tales hiding in this problem. Write a paragraph explaining what you can learn by playing with pathological matrices like the Hilbert Matrix.

Exercise 1.10.12. Now that you have QR and LU code we are going to use both of them! The problem is as follows:
We are going to find the polynomial of degree 4 that best fits the function

$$y = \cos(4t) + 0.1\varepsilon(t) \tag{1.118}$$

at 50 equally spaced points t between 0 and 1. Here we are using $\varepsilon(t)$ as a function that outputs normally distributed random white noise. In Python you will build y as `y = np.cos(4*t) + 0.1*np.random.randn(t.shape[0])`

Build the t vector and the y vector (these are your data). We need to set up the least squares problems $Ax = b$ by setting up the matrix A as we did in the other least squares curve fitting problems and by setting up the b vector using the y data you just built. Solve the problem of finding the coefficients of the best degree 4 polynomial that fits this data. Report the sum of squared error and show a plot of the data along with the best fit curve.

Exercise 1.10.13. Find the largest eigenvalue and the associated eigenvector of the matrix A WITHOUT using `np.linalg.eig()`. (Do not do this by hand either)

$$A = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 0 & 1 & 2 \\ 3 & 4 & 5 & 6 \end{pmatrix} \tag{1.119}$$

Exercise 1.10.14. It is possible in a matrix that the eigenvalues λ_1 and λ_2 are equal but with the corresponding eigenvectors not equal. Before you experiment with matrices of this sort, write a conjecture about what will happen to the power method in this case (look back to our proof in Exercise 1.8.7 of how the power method works). Now build several specific matrices where this is the case and see what happens to the power method.

Exercise 1.10.15. Will the power method fail, slow down, or be unaffected if one (or more) of the non-dominant eigenvalues is zero? Give sufficient mathematical evidence or show several numerical experiments to support your answer.

Theorem 1.10.1. *If A is a symmetric matrix with eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_n$ then $|\lambda_1| > |\lambda_2| > \cdots > |\lambda_n|$. Furthermore, the eigenvectors will be orthogonal to each other.*

Exercise 1.10.16 (The Deflation Method)**.** For symmetric matrices we can build an extension to the power method in order to find the second most dominant eigen-pair for a matrix A. Theorem 1.10.1 suggests the following method for finding the second dominant eigen-pair for a symmetric matrix. This method is called the **deflation method**.

- Use the power method to find the dominant eigenvalue and eigenvector.
- Start with a random unit vector of the correct shape.
- Multiplying your vector by A will *pull it toward* the dominant eigenvector. After you multiply, project your vector onto the dominant eigenvector and find the projection error.
- Use the projection error as the new approximation for the eigenvector (Why should we do this? What are we really finding here?)

Note that the deflation method is really exactly the same as the power method with the exception that we orthogonalize at every step. Hence, when you write your code expect to only change a few lines from your power method.

Write a function to find the second largest eigenvalue and eigenvector pair by putting the deflation method into practice. Test your code on a matrix A and compare against Python's `np.linalg.eig()` command. Your code needs to work on symmetric matrices of arbitrary size and you need to write test code that clearly shows the error between your calculated eigenvalue and Python's eigenvalue as well as your calculated eigenvector and 's eigenvector.

To guarantee that you start with a symmetric matrix you can use the following code.

```
import numpy as np
N = 40
A = np.random.randn(N,N)
# A = np.matrix(A) # No need for matrix class
A = np.transpose(A) @ A # why should this build a symmetric matrix
```

Exercise 1.10.17. (This concept for this problem is modified from (Meerschaert 2013)). The data is taken from NOAA and the National Weather Service with the specific values associated with La Crosse, WI.)

Floods in the Mississippi River Valleys of the upper Midwest have somewhat predictable day-to-day behaviour in that the flood stage today has high predictive power for the flood stage tomorrow. Assume that the flood stages are:

- Stage 0 (Normal): Average daily flow is below 90,000 ft^3/sec (cubic feet per second = cfs). This is the *normal* river level.
- Stage 1 (Action Level): Average daily flow is between 90,000 cfs and 124,000 cfs.
- Stage 2 (Minor Flood): Average daily flow is between 124,000 cfs and 146,000 cfs.
- Stage 3 (Moderate Flood): Average daily flow is between 146,000 cfs and 170,000 cfs.
- Stage 4 (Extreme Flood): Average daily flow is above 170,000 cfs.

The following table shows the probability of one stage transitioning into another stage from one day to the next.

	0 Today	**1 Today**	**2 Today**	**3 Today**	**4 Today**
0 Tomorrow	0.9	0.3	0	0	0
1 Tomorrow	0.05	0.7	0.4	0	0
2 Tomorrow	0.025	0	0.6	0.6	0
3 Tomorrow	0.015	0	0	0.4	0.8
4 Tomorrow	0.01	0	0	0	0.2

Mathematically, if s_k is the state at day k and A is the matrix given in the table above then the difference equation $s_{k+1} = As_k$ shows how a state will transition from day to day. For example, if we are currently in Stage 0 then

$$s_0 = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}. \tag{1.120}$$

We can interpret this as "*there is a probability of 1 that we are in Stage 0 today and there is a probability of 0 that we are in any other stage today.*"

If we want to advance this model forward in time then we just need to iterate. In our example, the state tomorrow would be $s_1 = As_0$. The state two days from now would be $s_2 = As_1$, and if we use the expression for s_1 we can simplify to $s_2 = A^2 s_0$.

1. Prove that the state at day n is $s_n = A^n s_0$.
2. If n is large then the steady state solution to the difference equation in part (1) is given exactly by the power method iteration that we have studied in this chapter. Hence, as the iterations proceed they will be pulled toward the dominant eigenvector. Use the power method to find the dominant eigenvector of the matrix A.
3. The vectors in this problem are called **probability vectors** in the sense that the vectors sum to 1 and every entry can be interpreted as a probability. Re-scale your answer from part (b) so that we can interpret the entries as probabilities. That is, ensure that the sum of the vector from part (b) is 1.
4. Interpret your answer to part (c) in the context of the problem. Be sure that your interpretation could be well understood by someone that does not know the mathematics that you just did.

Exercise 1.10.18. The LU factorization as we have built it in this chapter is not *smart* about the way that it uses the memory on your computer. In the LU factorization the 1's on the main diagonal do not actually need to be stored since we know that they will always be there. The zeros in the lower triangle of U do not need to be stored either. If you store the upper triangle values in the U matrix on top of the upper triangle of the L matrix then we still store a full matrix for L which contains both L and U simultaneously, but we do not have to store U separately and hence save computer memory. The modifications to the existing code for an LU solve is minimal – every time you call on an entry of the U matrix it is stored in the upper triangle of L instead. Write code to implement this new data storage idea and demonstrate your code on a few examples.

Exercise 1.10.19. In the algorithm that we used to build the QR factorization we built the R matrix as $R = Q^T A$ The trouble with this step is that it fills in a lot of redundant zeros into the R matrix – some of which may not be exactly zero. First explain why this will be the case. Then rewrite your QR factorization code so that the top triangle of R is filled with all of the projections (do this with a double `for` loop). Demonstrate that your code works on a few examples.

1.11 Projects

In this section we propose several ideas for projects related to numerical linear algebra. These projects are meant to be open ended, to encourage creative mathematics, to push your coding skills, and to require you to write and communicate your mathematics.

1.11.1 The Google Page Rank Algorithm

In this project you will discover how the Page Rank algorithm works to give the most relevant information as the top hit on a Google search.

Search engines compile large indexes of the dynamic information on the Internet so they are easily searched. This means that when you do a Google search, you are not actually searching the Internet; instead, you are searching the indexes at Google.

When you type a query into Google the following two steps take place:

1. Query Module: The query module at Google converts your natural language into a language that the search system can understand and consults the various indexes at Google in order to answer the query. This is done to find the list of relevant pages.
2. Ranking Module: The ranking module takes the set of relevant pages and ranks them. The outcome of the ranking is an ordered list of web pages such that the pages near the top of the list are most likely to be what you desire from your search. This ranking is the same as assigning a *popularity score* to each web site and then listing the relevant sites by this score.

This section focuses on the Linear Algebra behind the Ranking Module developed by the founders of Google: Sergey Brin and Larry Page. Their algorithm is called the *Page Rank algorithm*, and you use it every single time you use Google's search engine.

In simple terms: *A webpage is important if it is pointed to by other important pages.*

The Internet can be viewed as a directed graph (look up this term here on Wikipedia) where the nodes are the web pages and the edges are the hyperlinks between the pages. The hyperlinks into a page are called *in links*, and the ones pointing out of a page are called *out links*. In essence, a hyperlink from my page to yours is my endorsement of your page. Thus, a page with more recommendations must be more important than a page with a few links. However, the status of the recommendation is also important.

Let us now translate this into mathematics. To help understand this we first consider the small web of six pages shown in Figure 1.3 (a graph of the router level of the internet can be found here). The links between the pages are shown by arrows. An arrow pointing into a node is an *in link* and an arrow pointing out of a node is an *out link*. In Figure 1.3, node 3 has three out links (to nodes 1, 2, and 5) and 1 in link (from node 1).

We will first define some notation in the Page Rank algorithm:

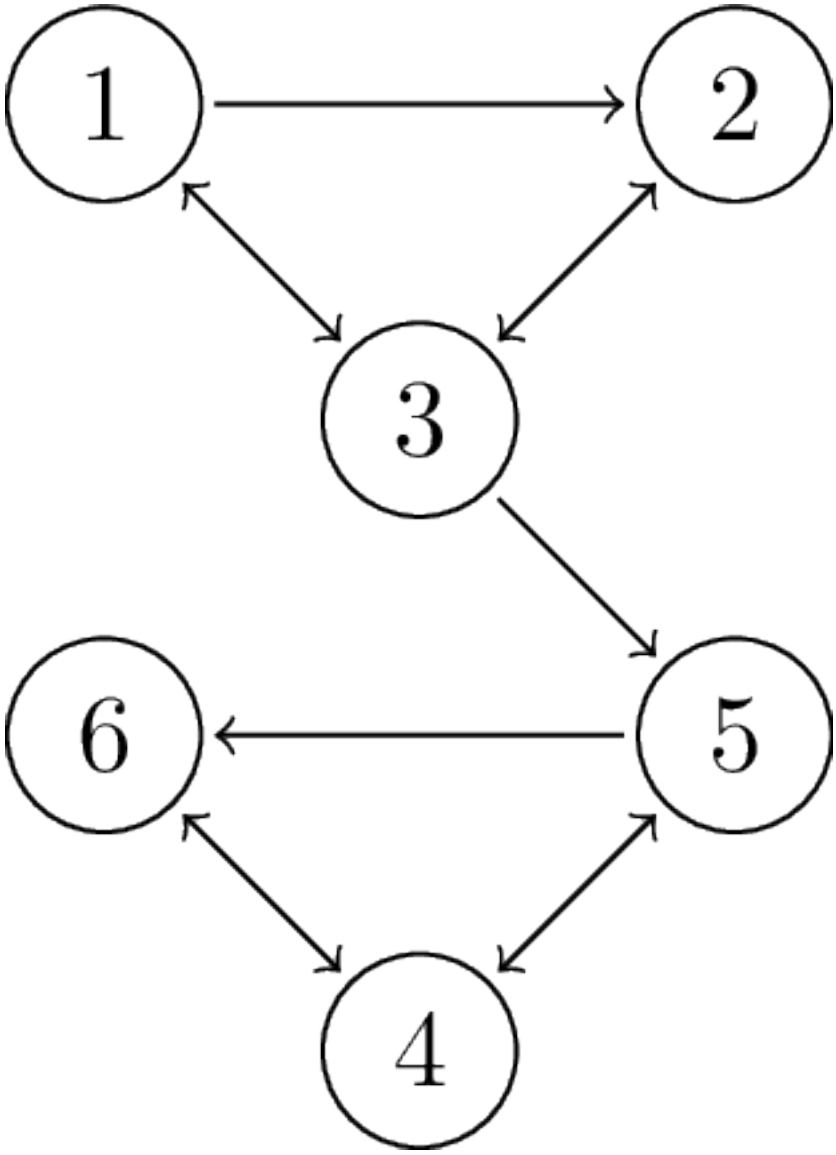

Figure 1.3: Example web graph.

- $|P_i|$ is the number of out links from page P_i
- H is the *hyperlink* matrix defined as

$$H_{ij} = \begin{cases} \frac{1}{|P_j|}, & \text{if there is a link from node } j \text{ to node } i \\ 0, & \text{otherwise} \end{cases} \tag{1.121}$$

  where the "i" and "j" are the row and column indices respectively.
- x is a vector that contains all of the Page Ranks for the individual pages.

The Page Rank algorithm works as follows:

1. Initialize the page ranks to all be equal. This means that our initial assumption is that all pages are of equal rank. In the case of Figure 1.3 we would take x_0 to be

$$x_0 = \begin{pmatrix} 1/6 \\ 1/6 \\ 1/6 \\ 1/6 \\ 1/6 \\ 1/6 \end{pmatrix}. \tag{1.122}$$

2. Build the hyperlink matrix.
   As an example we will consider node 3 in Figure 1.3. There are three out links from node 3 (to nodes 1, 2, and 5). Hence $H_{13} = 1/3$, $H_{23} = 1/3$, and $H_{53} = 1/3$ and the partially complete hyperlink matrix is

$$H = \begin{pmatrix} - & - & 1/3 & - & - & - \\ - & - & 1/3 & - & - & - \\ - & - & 0 & - & - & - \\ - & - & 0 & - & - & - \\ - & - & 1/3 & - & - & - \\ - & - & 0 & - & - & - \end{pmatrix} \tag{1.123}$$

3. The difference equation $x_{n+1} = Hx_n$ is used to iteratively refine the estimates of the page ranks. You can view the iterations as a person visiting a page and then following a link at random, then following a random link on the next page, and the next, and the next, etc. Hence we see that the iterations evolve exactly as expected for a difference equation.

Iteration	**New Page Rank Estimation**
0	x_0
1	$x_1 = Hx_0$

Iteration	New Page Rank Estimation
2	$x_2 = Hx_1 = H^2x_0$
3	$x_3 = Hx_2 = H^3x_0$
4	$x_4 = Hx_3 = H^4x_0$
$\vdots$	$\vdots$
k	$x_k = H^kx_0$

4. When a steady state is reached we sort the resulting vector x_k to give the page rank. The node (web page) with the highest rank will be the top search result, the second highest rank will be the second search result, and so on.

It does not take much to see that this process can be very time consuming. Think about your typical web search with hundreds of thousands of hits; that makes a square matrix H that has a size of hundreds of thousands of entries by hundreds of thousands of entries! The matrix multiplications alone would take many minutes (or possibly many hours) for every search! …but Brin and Page were pretty smart dudes!!

We now state a few theorems and definitions that will help us simplify the iterative Page Rank process.

Theorem 1.11.1. *If A is an $n \times n$ matrix with n linearly independent eigenvectors $v_1, v_2, v_3, \ldots, v_n$ and associated eigenvalues $\lambda_1, \lambda_2, \lambda_3, \ldots, \lambda_n$ then for any initial vector $x \in \mathbb{R}^n$ we can write A^kx as*

$$A^kx = c_1\lambda_1^kv_1 + c_2\lambda_2^kv_2 + c_3\lambda_3^kv_3 + \cdots c_n\lambda_n^kv_n \tag{1.124}$$

where $c_1, c_2, c_3, \ldots, c_n$ are the constants found by expressing x as a linear combination of the eigenvectors.
Note: We can assume that the eigenvalues are ordered such that $|\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \cdots \geq |\lambda_n|$.

Exercise 1.11.1. Prove the preceding theorem.

Definition 1.11.1. A **probability vector** is a vector with entries on the interval $[0, 1]$ that add up to 1. A **stochastic matrix** is a square matrix whose columns are probability vectors.

Theorem 1.11.2. *If A is a stochastic $n \times n$ matrix then A will have n linearly independent eigenvectors. Furthermore, the largest eigenvalue of a stochastic matrix will be $\lambda_1 = 1$ and the smallest eigenvalue will always be non-negative: $0 \leq |\lambda_n| < 1$.*

Some of the following tasks will ask you to prove *a statement or a theorem. This means to clearly write all of the logical and mathematical reasons why the statement is true. Your proof should be absolutely crystal clear to anyone with a similar mathematical background …if you are in doubt then have a peer from a different group read your proof to you .*

Exercise 1.11.2. Finish writing the hyperlink matrix H from Figure 1.3.

Exercise 1.11.3. Write code to implement the iterative process defined previously. Make a plot that shows how the rank evolves over the iterations.

Exercise 1.11.4. What must be true about a collection of n pages such that an $n \times n$ hyperlink matrix H is a stochastic matrix.

Exercise 1.11.5. The statement of the next theorem is incomplete, but the proof is given to you. Fill in the blank in the statement of the theorem and provide a few sentences supporting your answer.

Theorem 1.11.3. *If A is an $n \times n$ stochastic matrix and x_0 is some initial vector for the difference equation $x_{n+1} = Ax_n$, then the steady state vector is*

$$x_{equilib} = \lim_{k \to \infty} A^k x_0 = \underline{\qquad\qquad}. \tag{1.125}$$

Proof:

First note that A is an $n \times n$ stochastic matrix so from Theorem 1.11.2 we know that there are n linearly independent eigenvectors. We can then substitute the eigenvalues from Theorem 1.11.2 in Theorem 1.11.1. Noting that if $0 < \lambda_j < 1$ we have $\lim_{k \to \infty} \lambda_j^k = 0$ the result follows immediately.

Exercise 1.11.6. Discuss how Theorem 1.11.3 greatly simplifies the PageRank iterative process described previously. In other words: there is no reason to iterate at all. Instead, just find … what?

Exercise 1.11.7. Now use the previous two problems to find the resulting PageRank vector from the web in Figure 1.3? Be sure to rank the pages in order of importance. Compare your answer to the one that you got in problem 2.

Exercise 1.11.8. Consider the web in Figure 1.4.

1. Write the H matrix and find the initial state x_0,
2. Find steady state PageRank vector using the two different methods described: one using the iterative difference equation and the other using Theorem 1.11.3 and the dominant eigenvector.
3. Rank the pages in order of importance.

Exercise 1.11.9. One thing that we did not consider in this version of the Google Page Rank algorithm is the random behaviour of humans. One, admittedly slightly naive, modification that we can make to the present algorithm is to assume that the person surfing the web will randomly jump to any other page in the web at any time. For example, if someone is on page 1 in Figure 1.4 then they could randomly jump to any page 2 - 8. They also have links to pages 2, 3, and 7. That is a total of 10 possible next steps for the web surfer. There is a 2/10 chance of heading to page 2. One of those is following the link from page 1 to page 2 and the other is a random jump to page 2 without following the link. Similarly, there is a 2/10 chance of heading to page 3, 2/10 chance of heading to page 7, and a 1/10 chance of randomly heading to any other page.

Implement this new algorithm, called the *random surfer algorithm*, on the web in Figure 1.4. Compare your ranking to the non-random surfer results from the previous problem.

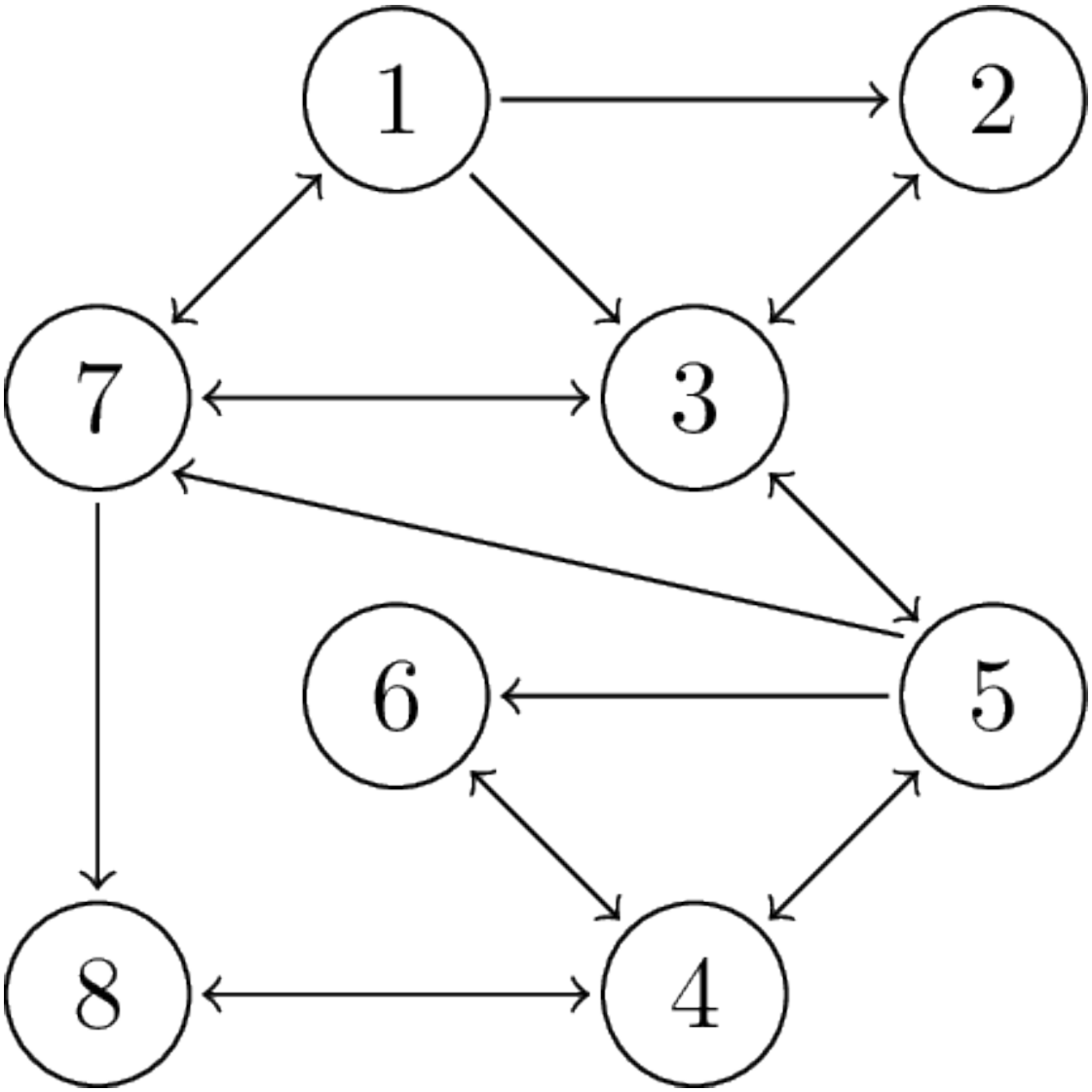

Figure 1.4: A second example web graph.

1.11.2 Alternative Methods For Solving $Ax = b$

Throughout most of the linear algebra chapter we have studied ways to solve systems of equations of the form $Ax = b$ where A is a square $n \times n$ matrix, $x \in \mathbb{R}^n$, and $b \in \mathbb{R}^n$. We have reviewed by-hand row reduction and learned new techniques such as the LU decomposition and the QR decomposition – all of which are great in their own right and all of which have their shortcomings.

Both LU and QR are great solution techniques and they generally work very very well. However (no surprise), we can build algorithms that will *usually* be faster!

In the following new algorithms we want to solve the linear system of equations

$$Ax = b \tag{1.126}$$

but in each we will do so iteratively by applying an algorithm over and over until the algorithm converges to an approximation of the solution vector x. Convergence here means that $\|A\ x - b\|$ is less than some pre-determined tolerance.

Method 1: Start by "factoring"[7] the matrix A into $A = L + U$ where L is a lower-triangular matrix and U is an upper-triangular matrix. Take note that this time we will not force the diagonal entries of L to be 1 like we did in the classical LU factorization . The U in the factorization $A = L + U$ is an upper-triangular matrix where the entries on the main diagonal are exactly 0.

Specifically,

$$\begin{aligned} A = L + U = &\begin{pmatrix} a_{00} & 0 & 0 & \cdots & 0 \\ a_{10} & a_{11} & 0 & \cdots & 0 \\ a_{20} & a_{21} & a_{22} & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n0} & a_{n1} & a_{n2} & \cdots & a_{n-1,n-1} \end{pmatrix} \\ &+ \begin{pmatrix} 0 & a_{01} & a_{02} & \cdots & a_{0,n-1} \\ 0 & 0 & a_{12} & \cdots & a_{1,n-1} \\ 0 & 0 & 0 & a_{23} & \cdots \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & & 0 \end{pmatrix}. \end{aligned} \tag{1.127}$$

As an example,

$$\begin{pmatrix} 2 & 3 & 4 \\ 5 & 6 & 7 \\ 8 & 9 & 1 \end{pmatrix} = \begin{pmatrix} 2 & 0 & 0 \\ 5 & 6 & 0 \\ 8 & 9 & 1 \end{pmatrix} + \begin{pmatrix} 0 & 3 & 4 \\ 0 & 0 & 7 \\ 0 & 0 & 0 \end{pmatrix}. \tag{1.128}$$

[7] Technically speaking we should not call this a "factorization" since we have not split the matrix A into a product of two matrices. Instead we should call it a "partition" since in number theory we call the process of breaking an integer into the sum of two integers is called a "partition." Even so, we will still use the word factorization here for simplicity.

After factoring the system of equations can be rewritten as

$$Ax = b \implies (L + U)x = b \implies Lx + Ux = b. \tag{1.129}$$

Moving the term Ux to the right-hand side gives $Lx = b - Ux$, and if we solve for the unknown x we get $x = L^{-1}(b - Ux)$.

Of course we would never (*ever*!) actually compute the inverse of L, and consequently we have to do something else in place of the matrix inverse. Stop and think here for a moment. We have run into this problem earlier in this chapter and you have some code that you will need to modify for this job (but take very careful note that the L matrix here does not quite have the same structure as the L matrix we used in the past). Moreover, notice that we have the unknown x on both sides of the equation. Initially this may seem like nonsense, but if we treat this as an iterative scheme by first making a **guess** about x and then iteratively find better approximations of solutions via the difference equation

$$x_{k+1} = L^{-1}(b - Ux_k) \tag{1.130}$$

we may, under moderate conditions on A, quickly be able to approximate the solution to $Ax = b$. The subscripts in the iterative scheme represent the iteration number. Hence,

$$x_1 = L^{-1}(b - Ux_0) \tag{1.131}$$

$$x_2 = L^{-1}(b - Ux_1) \tag{1.132}$$

$$\vdots \tag{1.133}$$

What we need to pay attention to is that the method is not guaranteed to converge to the actual solution to the equation $Ax = b$ unless some conditions on A are met, and you will need to experiment with the algorithm to come up with a conjecture about the appropriate conditions.

Method 2: Start by factoring the matrix A into $A = L + D + U$ where L is strictly lower-triangular (0's on the main diagonal and in the entire upper triangle), D is a diagonal matrix, and U is a strictly upper-triangular matrix (0's on the main diagonal and in the entire lower triangle). In this new factorization, the diagonal matrix D simply contains the entries from the main diagonal of A. The L matrix is the lower triangle of A, and the U matrix is the upper triangle of A.

Considering the system of equations $Ax = b$ we get

$$(L + D + U)x = b \tag{1.134}$$

and after simplifying, rearranging, and solving for x we get $x = D^{-1}(b - Lx - Ux)$. A moment's reflection should reveal that the inverse of D is really easy to find (no

heavy-duty linear algebra necessary) if some mild conditions on the diagonal entries of A are met. Like before there is an x on both sides of the equation, but if we again make the algorithm iterative we can get successive approximations of the solution with

$$x_{k+1} = D^{-1}(b - Lx_k - Ux_k). \tag{1.135}$$

Your Tasks:

1. Pick a small (larger than 3×3) matrix and an appropriate right-hand side b and work each of the algorithms by hand. You do not need to write this step up in the final product, but this exercise will help you locate where things may go wrong in the algorithms and what conditions we might need on A in order to get convergent sequences of approximate solutions.

2. Build Python functions that accept a square matrix A and complete the factorizations $A = L + U$ and $A = L + D + U$.

3. Build functions to implement the two methods and then demonstrate that the methods work on a handful of carefully chosen test examples. As part of these functions you need to build a way to deal with the matrix inversions as well as build a stopping rule for the iterative schemes. Hint: You should use a `while` loop with a proper logical condition. Think carefully about what we are finding at each iteration and what we can use to check our accuracy at each iteration. It would also be wise to write your code in such a way that it checks to see if the sequence of approximations is diverging.

4. Discuss where each method might fail and then demonstrate the possible failures with several carefully chosen examples. Stick to small examples and work these out by hand to clearly show the failure.

5. Iterative methods such as these will produce a sequence of approximations, but there is no guarantee that either method will actually produce a convergent sequence. Experiment with several examples and propose a condition on the matrix A which will likely result in a convergent sequence. Demonstrate that the methods fail if your condition is violated and that the methods converge if your condition is met. Take care that it is tempting to think that your code is broken if it does not converge. The more likely scenario is that the problem that you have chosen to solve will result in a non-convergent sequence of iterations, and you need to think and experiment carefully when choosing the example problems to solve. One such convergence criterion has something to do with the diagonal entries of A relative to the other entries, but that does not mean that you should not explore other features of the matrices as well (I cannot give you any more hints than that). This task is not asking for a proof; just a conjecture and convincing numerical evidence that the conjecture holds. The actual proofs are beyond the scope of this project and this course.

6. Devise a way to demonstrate how the time to solve a large linear system $Ax = b$ compares between our two new methods, the LU algorithm, and the QR algorithm that we built earlier in the chapter. Conclude this demonstration with appropriate plots and ample discussion.

You need to do this project without the help of your old buddy Google. All code must be originally yours or be modified from code that we built in class. You can ask Google how Python works with matrices and the like, but searching directly for the algorithms (which are actually well-known, well-studied, and named algorithms) is not allowed.

Finally, solving systems of equations with the `|np.linalg.solve()` command can only be done to verify or check your answer(s).

Meerschaert, Mark. 2013. *Mathematical Modeling.* 4th edition. Academic Press.